

Population Protocols over Ordered Agents

Michael Blondin ✉ 

Département d'informatique, Université de Sherbrooke, Canada

Michaël Cadilhac ✉ 

DePaul University, Chicago, USA

Benjamin Courchesne ✉ 

Département d'informatique, Université de Sherbrooke, Canada

Lucie Guillou ✉ 

MPI for Software Systems, Kaiserslautern, Germany

Corto Mascle ✉ 

MPI for Software Systems, Kaiserslautern, Germany

Isa Vialard ✉ 

MPI for Software Systems, Saarbrücken, Germany

Abstract

Population protocols are a distributed computation model in which a collection of anonymous, finite-state agents interact in randomly chosen pairs and update their states according to a fixed transition function. The computation is defined by the eventual stabilization of the population to a consensus that represents the output. In practice, it is natural to allow each agent to carry a unique identifier and compare it with that of another agent before interacting. We model this extension by having agents be totally ordered and interactions between two agents to be fireable only if their pair of identifiers falls in some condition set. For instance, $\text{PP}[\leq]$ allows for two agents to interact only if the first one appears before the second one.

We study population protocols over ordered agents $\text{PP}[\mathcal{N}]$ where $\mathcal{N}$ is a set of predicates available to restrict transition firing. We also study $\text{IO-PP}[\mathcal{N}]$, the *immediate observation* fragment of $\text{PP}[\mathcal{N}]$ where only one agent changes state per interaction. Our main result is that $\text{IO-PP}[\leq]$ recognizes exactly the unambiguous star-free languages, which admits many other characterizations, such as two-variable first-order logic or two-way deterministic partially-ordered automata. We also provide a logic and an automaton model that fits in $\text{PP}[\leq]$. We further show that if the successor predicate appears in a set $\mathcal{N}$ of $\text{NSPACE}(n)$ -computable predicates, then $\text{IO-PP}[\mathcal{N}] = \text{PP}[\mathcal{N}] = \text{NSPACE}(n)$. Finally, we investigate the problem of deciding whether a given population protocol always stabilizes to a consensus. While this problem is decidable for unordered population protocols, we show that this is undecidable already for $\text{PP}[\leq]$ and $\text{IO-PP}[+1]$, but conditionally decidable for $\text{IO-PP}[\leq]$.

2012 ACM Subject Classification Theory of computation $\rightarrow$ Formal languages and automata theory; Theory of computation $\rightarrow$ Distributed computing models; Theory of computation $\rightarrow$ Logic and verification

Keywords and phrases Population protocols, First-order logic, Partially-ordered automata, Unambiguous star-free languages

Funding *Michael Blondin*: supported by a Discovery Grant from the Natural Sciences and Engineering Research Council of Canada (NSERC).

Acknowledgements We thank the anonymous reviewers of ICALP 2026 for their thorough reading and valuable suggestions.

Throughout the article, we provide hyperlinks for navigating between statements in the main text and proofs in the appendix. To go to the proof of a statement, click “ $\downarrow$ ” in the left margin, and then click “ $\uparrow$ ” to go back to the statement in the main text.

Contents

1	Introduction	1
2	Preliminaries	3
3	Semantic restrictions of population protocols: a toolbox	7
3.1	Semi-deciders	7
3.2	Protocols with stabilizing inputs	8
4	Expressiveness of IO-PP[$<$]	9
4.1	IO-PP[$<$] $\subseteq$ DA	9
4.2	DA $\subseteq$ IO-PP[$<$]	10
5	Expressiveness of PP[$<$]	11
5.1	First-order logic over word intervals	11
5.2	Partially-ordered Parikh automata	12
5.3	The regular languages of PP[$<$]	14
6	Expressiveness of IO-PP[$\mathcal{N}$] and PP[$\mathcal{N}$] when successor is available	14
6.1	NSPACE(n) $\subseteq$ IO-PP[+1]	15
7	Decidability of checking whether a population protocol is a decider	16
7.1	The syntax of PP[$<$], IO-PP[+1] and PP[+1] deciders are undecidable	16
7.2	The syntax of IO-PP[$<$] deciders is decidable, conditionally	17
8	Open questions	17
A	Appendix	21
A.1	Missing proofs from Section 3.1	21
A.2	Missing proofs from Section 3.2	21
A.3	Missing proofs from Section 4.1	24
A.4	Missing proofs from Section 4.2	26
A.5	Missing proofs from Section 5.1	26
A.6	Missing proofs from Section 5.2	27
A.7	Missing proofs from Section 5.3	28
A.8	Missing proofs from Section 6.1	29
A.9	Missing proofs from Section 7.1	32
A.10	Missing proofs from Section 7.2	34

1 Introduction

Population protocols form a well-established model of distributed computing where anonymous agents, with very limited individual computational power, work collectively to achieve a common task [3]. In this model, an input is scattered among agents that interact pairwise and must take a decision by reaching a consensus that is stable, that is, agents must eventually all agree on the output (“consensus”) and stop changing their mind (“stable”). Population protocols provide a theoretical framework for reasoning about a wide range of distributed systems, including networks of mobile sensors, chemical reaction networks, and social networks [5, 10, 11].

To familiarize the reader with population protocols, we present a classical protocol for the task of majority voting. A population consists of n agents (who are not aware of n), each carrying a state from a finite set. Here, agents start with either a or b as their state. The population aims at collectively determining whether there are initially more a ’s than b ’s. At each discrete moment, a pair of agents is chosen arbitrarily and their respective state, from $Q = \{a, b, \underline{a}, \underline{b}\}$, is updated according to these rules:

<i>active to passive</i>	<i>propagation of winning side</i>	<i>tiebreaker</i>
$a, b \rightarrow \underline{a}, \underline{b}$	$a, \underline{b} \rightarrow a, \underline{a}$ $b, \underline{a} \rightarrow b, \underline{b}$	$\underline{a}, \underline{b} \rightarrow \underline{b}, \underline{b}$

Since agents are unordered, each rule $p, q \rightarrow p', q'$ also stands for $q, p \rightarrow q', p'$. Here are three possible executions of the protocol starting from three different “inputs,” that is, assignments of an initial state a or b to each agent:

$$\begin{aligned}
 aaabb &\rightarrow aa\underline{abb} \rightarrow \underline{aaabb} \rightarrow \underline{aaaa}b \rightarrow \underline{aaaaa}, \\
 aabbb &\rightarrow \underline{a}b\underline{bb}b \rightarrow \underline{aa}b\underline{bb} \rightarrow \underline{aabbb} \rightarrow \underline{abbb}b \rightarrow \underline{abbbb}, \rightarrow \underline{bbbb}, \\
 aabb &\rightarrow \underline{a}b\underline{bb} \rightarrow \underline{abb}b \rightarrow \underline{abbb} \rightarrow \underline{bbbb}.
 \end{aligned}$$

Assuming fair scheduling (e.g., choosing agents uniformly at random), one can show that the population stabilizes (almost surely) to the correct outcome: if there is a majority of a ’s initially, then agents eventually remain in $\{a, \underline{a}\}$, otherwise they eventually remain in $\{b, \underline{b}\}$.

As agents and rules are unordered, each configuration can be seen as a multiset $\mathbf{c}: Q \rightarrow \mathbb{N}$ where $\mathbf{c}(q)$ indicates the number of agents in state q . It is known that population protocols compute precisely the subsets of $\mathbb{N}^Q$ that are semilinear [4], or, equivalently, that are definable in Presburger arithmetic (the first-order theory of the naturals with order and addition). In particular, majority voting amounts to computing the predicate $\varphi(\mathbf{c}) = \mathbf{c}(a) > \mathbf{c}(b)$.

From a modeling perspective, the fact that agents are unordered is meant to correspond to a situation in which agents are replicated and anonymous entities, and hence have no identifiers and are indistinguishable. Yet, it is natural to allow replicated agents to be totally ordered, e.g., they could be devices with a unique identifier, such as a serial number, stored in read-only memory. In this context, interactions may depend on the relationships between these identifiers.

From the perspective of automata theory, this corresponds to considering population protocols where configurations are *words* rather than multisets. Standard population protocols can be seen as computing commutative properties of words, such as $|w|_a > |w|_b$, while the word setting additionally allows for noncommutative properties, such as “the middle agent has an a ” or “agents strictly alternate between a and b .”

Contribution

Motivated by the above, we propose to study population protocols with totally-ordered agents. The class of population protocols so defined, $\text{PP}[\mathcal{N}]$, is parameterized by a set $\mathcal{N}$ of predicates over positions that can be used to restrict transition firings. Central to our study is the class $\text{PP}[<]$, in which a transition $p, q \xrightarrow{\quad} r, s$ can only be applied to two agents in respective states p and q if the first agent appears *before* the second agent. We consider a well-studied restriction called *immediate observation* [4] where an interaction can only update a single agent, called the “observer.”

Our main result, Theorem 12, establishes that $\text{IO-PP}[<]$ has the same expressive power as unambiguous star-free languages, an important subclass of regular languages that admits a trove of characterizations (see [35] for a lovely survey on the pervasiveness of this class in automata theory). Hence, $\text{IO-PP}[<]$ is the class of languages captured by these formalisms over finite words:

- partially-ordered unambiguous automata [26];
- partially-ordered two-way deterministic automata [33];
- $\text{LTL}[\mathbf{F}^{-1}, \mathbf{F}]$: linear temporal logic with past and future operators [15];
- $\text{FO}^2[<]$: the two-variable fragment of first-order logic with order [36];
- $\Delta_2[<]$: the intersection of the $\exists^*\forall^*$ and $\forall^*\exists^*$ fragments of first-order logic with order [28];
- languages recognized by finite monoids from the variety DA [32].

This provides the first characterization of this class in terms of distributed computing, rather than automata, logic, or algebra. We will provide in Example 3 an example of a $\text{PP}[<]$ protocol whose language is not regular, and therefore not expressible by $\text{IO-PP}[<]$ protocols.

In addition, we explore systematically the classes induced by our definitions:

- In Section 3, we provide a toolbox to study population protocols over ordered agents $\text{PP}[\mathcal{N}]$, regardless of $\mathcal{N}$. We study protocols that need to stabilize only if they reach a positive consensus, which we call *semi-deciders*, as opposed to protocols that always stabilize to a consensus, dubbed *deciders*. We also refine the technology of protocols with *stabilizing inputs*, studied in [31], which enables a form of composition between protocols.
- In Section 4, we prove the aforementioned characterization of $\text{IO-PP}[<]$, and in Section 5, we provide a natural logic and an automaton model expressible in $\text{PP}[<]$. In this latter section, we fall short of showing exact characterizations, but provide conjectures based on our new models.
- In Section 6, we explore the expressiveness of $\text{IO-PP}[\mathcal{N}]$ and $\text{PP}[\mathcal{N}]$ when the successor predicate is available, that is, when transitions can be restricted to fire only if they act on two adjacent agents. We show that if all the predicates of $\mathcal{N}$ are $\text{NSPACE}(n)$ computable, then $\text{IO-PP}[\mathcal{N}] = \text{PP}[\mathcal{N}] = \text{NSPACE}(n)$ — this is arguably less surprising than our main result on $\text{IO-PP}[<]$, as the successor allows for the left-to-right propagation of information.
- Finally, since protocols are only well-behaved when they are deciders, and since this property is semantic, we explore in Section 7 whether we can check if a given population protocol is a decider. We thus ask if the *syntax* of deciders is decidable — this is sometimes called the *well-specification* problem. We show that it is undecidable for $\text{IO-PP}[+1]$, $\text{PP}[+1]$, and $\text{PP}[<]$, and conditionally decidable for $\text{IO-PP}[<]$.

Related work

Our model is closely related to the *community protocols* of Guerraoui and Ruppert [19]. These are population protocols where each agent has a unique identifier; each agent can

store a constant number of identifiers; and interactions depend on these identifiers but only with respect to their relative order. The motivation of Guerraoui and Ruppert was to devise an extension of population protocols, as mild as possible, which would be fault-tolerant. They proved that community protocols can decide languages from $\text{NSPACE}(n \log n)$ while tolerating Byzantine failures of a constant number of agents [19]. The unique identifier of each agent is considered to be stored in read-only memory, as in real-world low-cost chips, and so exempt from Byzantine failures. Our model corresponds to community protocols where each agent has a *single immutable* register initialized to its unique identifier.

Bournez, Cohen, and Rabie introduce *homonym protocols*, a parameterized restriction of community protocols, where the n agents have $f(n)$ identifiers [8]. The cases of $f(n) = 1$ and $f(n) = n$ are respectively population protocols (everyone has the same identifier) and community protocols (there are as many identifiers as agents). Identifiers are from $[0..f(n)-1]$ and agents can compare them with respect to $x < y$, $x = y$, $x = y + 1$ and $x = 0$.

In [17], Gańczorz et al. introduce *selective population protocols* as an extension of population protocols where the state space is partitioned into finitely many “groups,” and where an interaction picks at random an initiator in state s , and then a responder in state s' from the group of s , provided it is nonempty. This is a powerful model that allows for zero-tests, i.e., checking whether no agent holds a certain state. Thus, selective population protocols are orthogonal to our model. However, the authors dedicate a section to the median problem: $\bigcup_{n \geq 0} \Sigma^n a \Sigma^n$. They show that if selective population protocols are extended with the possibility of comparing keys, then they can solve the median problem in time $\mathcal{O}(\log^4 n)$. The authors further provide a short proof that, without leveraging the “selective” aspect of their model (i.e., groups and zero-tests), any population protocol for the median problem must work in expected time $\Omega(n)$. This latter setting, only briefly discussed in [17], corresponds to our model.

Further extensions include *mediated population protocols* [25], where communication edges have an internal state; *population protocols with unordered data* [6], where the input alphabet is infinite; and *population protocols for graph class identification problems* [40, 2], where agents aim at determining whether the communication topology satisfies some property. For other work on immediate-observation protocols, see [14, 38, 12, 37].

2 Preliminaries

Automata and logic. We assume some familiarity with formal languages, automata theory and logic over finite words (e.g., see the textbook [12]). For a word $w \in \Sigma^*$, we write $w[i]$ for the i -th letter of w , starting at 1, and $w[i..j]$ for the infix $w[i]w[i+1] \cdots w[j]$. For $\sigma \in \Sigma$, we write $|w|_\sigma$ for the number of occurrences of σ in w .

We write FO to denote first-order logic over words, where quantifiers range over positions, that is, the set $\{1, \dots, |w|\}$ for a given word w , and where $a(x)$ holds with respect to w iff $w[x] = a$ (see, e.g., [12, Chap. 8] for formal definitions). We write FO[<] for the extension of FO with the numerical predicate <, that allows to test whether $x < y$ for two positions x and y . For example, the sentence $\varphi = (\exists x)(\forall y)[a(x) \wedge ((x < y) \rightarrow b(y))]$ describes the language Σ^*ab^* . By abuse of notation, FO[<] stands for both the set of syntactic sentences and for the class of languages described by these sentences. It is well known that FO[<] is the class of star-free (regular) languages.

We write $\Sigma_i[<]$ (resp. $\Pi_i[<]$) for the fragment of FO[<] of sentences in prenex normal form with i blocks of alternating quantifiers starting with $\exists$ (resp. $\forall$). For example, Σ^*ab^* belongs to $\Sigma_2[<]$ due to the form of our example φ . We let $\Delta_i[<] = \Sigma_i[<] \cap \Pi_i[<]$.

Population protocols over ordered agents. A population protocol (PP) describes how a totally-ordered set of finite-state agents interact and reach a decision about their overall initial states. Interactions can happen between any pair of agents, and predicates are used to restrict how transitions can be taken, based on the position of the agents in the order.

(*Syntax.*) We extend classical PPs to allow for transitions to carry a test on the *positions* of the totally-ordered agents. Let $\mathcal{N} \subseteq 2^{\mathbb{N} \times \mathbb{N}}$ be any set, whose elements we call *numerical predicates*: these will be used as the allowed tests on a transition.¹ We let $\mathbf{true} = \mathbb{N}^2$ be the always-true predicate. The set $\text{PP}[\mathcal{N}]$ of PPs over $\mathcal{N}$ is the set of transition systems (Q, Σ, O, Δ) where Q is a finite set of *states*, $\Sigma \subseteq Q$ is a distinguished subset of *initial* states, $O: Q \rightarrow \{\top, \perp\}$ maps each state to an *opinion*, and $\Delta \subseteq Q^2 \times (\mathcal{N} \cup \{\mathbf{true}\}) \times Q^2$ is a set of *transitions*. An element of Δ is denoted $q_1, q_2 \xrightarrow{P} q_3, q_4$, expressing, intuitively, that if two distinct agents meet, the first being in position i and state q_1 , the second in position j and state q_2 , such that $(i, j) \in P$, then the first agent changes its state to q_3 and the second to q_4 . We use Σ for the set of initial states as we would like to see such protocols as language acceptors: The initial configuration should be thought of as an input word over alphabet Σ .

If $\mathcal{N} = \{P_1, P_2, \dots\}$, we write $\text{PP}[P_1, P_2, \dots]$ for $\text{PP}[\mathcal{N}]$. Classical PPs can be seen as the class $\text{PP}[\emptyset]$, recalling that we assume that $\mathbf{true}$ is always available as a numerical predicate. Our main interest is in the class $\text{PP}[<]$, where $<$ is seen as the set of pairs (i, j) with $i < j$, but we will study more expressive predicates in Sections 3 and 6.

A PP is *immediate-observation* if at most one agent changes state in each interaction, i.e., every transition is of the form $a, b \xrightarrow{P} a, c$ or $a, b \xrightarrow{P} c, b$. We write $\text{IO-PP}[\mathcal{N}]$ for the class of protocols in $\text{PP}[\mathcal{N}]$ that are immediate-observation.

(*Semantics.*) Since our agents are totally ordered, we define the *configuration* of a system as a word of Q^+ . *Initial configurations* are words of Σ^+ . Let u and v be two configurations of the same length, we say that u *leads to* v , denoted $u \rightarrow v$, if the two configurations are equal except at potentially two distinct positions i and j , and there is a transition $u[i], u[j] \xrightarrow{P} v[i], v[j] \in \Delta$ with $(i, j) \in P$. We let $\rightarrow^*$ be the reflexive transitive closure of $\rightarrow$.

Consensus and stability. With w a configuration, let $O(w) \in \{\perp, \top\}$ be the common opinion of all states appearing in w , if there is one; otherwise $O(w)$ is undefined. A configuration w is a *b-consensus* if $O(w) = b$. It is further *b-stable* if $w \rightarrow^* v$ implies that v is a *b-consensus*.

► **Example 1.** We give an example of a protocol in $\text{PP}[<]$. Consider the transition system $\mathcal{P} = (\{a, b, q_\perp\}, \{a, b\}, O, \Delta)$, with $O(a) = O(b) = \top$, $O(q_\perp) = \perp$, and transitions $b, a \xrightarrow{\text{true}} q_\perp, q_\perp$ and $q_\perp, _ \xrightarrow{\text{true}} q_\perp, q_\perp$ with “ $_$ ” standing for any state. Every input configuration belongs to $(a + b)^+$ and is therefore a $\top$ -consensus (all agents output $\top$ initially). However, such configurations need not be $\top$ -stable. For instance, starting from ba we can apply the rule $b, a \xrightarrow{\text{true}} q_\perp, q_\perp$ and obtain $q_\perp q_\perp$, which is $\perp$ -stable. In contrast, every configuration in $a^* b^*$ is $\top$ -stable: it is a $\top$ -consensus and no transition is enabled.

Language of a PP. Consider an infinite sequence $w_0 \rightarrow w_1 \rightarrow w_2 \rightarrow \dots$ of configurations, which we call a *run*. We say that it is *fair* if for every w_i that appears infinitely often, each configuration of $\{v \mid w_i \rightarrow v\}$ appears infinitely often as well. By induction, fairness guarantees that each configuration of $\{v \mid w_i \rightarrow^* v\}$ appears infinitely often as well. Intuitively, fairness ensures that reachable configurations cannot be avoided forever (in a probabilistic

¹ These are sometimes called *uniform* numerical predicates in the literature, to emphasize the fact that they do not depend on the *total* number of agents. A natural predicate that is not uniform is $\mathbf{max}(x)$ which is true if x is the position of the last agent. This technical difference will not impact our results.

setting, where the scheduling induces a probability distribution on the runs, the resulting runs are almost surely fair). We will assume that any configuration can be extended into a run, and so into a fair run, by implicitly adding “no operation” transitions.

The *language* of a PP $\mathcal{P}$ is the set $L(\mathcal{P})$ of initial configurations from which there is a fair run that visits a $\top$ -stable configuration. Naturally, an initial configuration can be the origin of a fair run visiting a $\top$ -stable configuration, another run visiting a $\perp$ -stable configuration, or even a run that visits no stable configurations. We single out PPs that have more crisp behaviors. We say that a PP is a *decider* when for all $w \in \Sigma^*$, there is a $b \in \{\top, \perp\}$ such that *all* fair runs from w visit a b -stable configuration (i.e., for all finite runs from w to some configuration u , there is a path from u to a b -stable configuration). This is usually called *well-specified* in the literature and we justify our nomenclature in Section 3.1. We say that $L(\mathcal{P})$ is PP[$\mathcal{N}$]-decidable or IO-PP[$\mathcal{N}$]-decidable with the obvious meaning. We will identify PP[$\mathcal{N}$] and IO-PP[$\mathcal{N}$] with the class of languages decidable by these protocols.

Note that since population protocols are ill-defined when no agents are present, we adopt the convention, when working with their languages, to disregard the empty word.

► **Example 2.** Consider the protocol of PP[$<$] from Example 1. We show that it decides the language a^*b^* . Every input in a^*b^* is already $\top$ -stable. Conversely, if an input word is not in a^*b^* , then it contains the factor ba . Hence, in any fair run, the transition $(b, a) \rightarrow (q_\perp, q_\perp)$ is eventually executed. From that point on, every remaining agent eventually interacts with a $q_\perp$ -agent and is converted to $q_\perp$, so the run reaches a configuration in $q_\perp^*$, which is $\perp$ -stable.

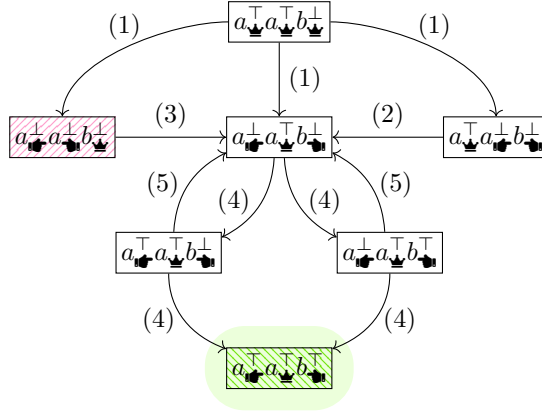
► **Example 3.** Consider the *median language* $L = \bigcup_{n \in \mathbb{N}} \Sigma^n a \Sigma^n$. Let us describe a protocol $\mathcal{P} = (Q, \Sigma, O, \Delta)$ that decides L . The states are defined as $Q = \Sigma \times \{\blacktriangleleft, \blacktriangle, \blacktriangleright\} \times \{\perp, \top\}$. The components respectively represent the input letter; a belief on whether the center is on the right, here, or on the left; and a belief on the output.

We identify input a with state $(a, \blacktriangle, \top)$, and each input $\sigma \neq a$ with $(\sigma, \blacktriangle, \perp)$. We set $O((x, y, z)) = z$. The set Δ is defined by these rules, each describing a family of transitions:

<i>Population halving</i>			
(1)	$(x, \blacktriangle, z), (x', \blacktriangle, z')$	$\searrow$	$(x, \blacktriangleleft, \perp), (x', \blacktriangleright, \perp)$
<i>Center finding</i>			
(2)	$(x, \blacktriangle, z), (x', \blacktriangleleft, z')$	$\searrow$	$(x, \blacktriangleleft, \perp), (x', \blacktriangle, x' = a)$
(3)	$(x, \blacktriangleright, z), (x', \blacktriangle, z')$	$\searrow$	$(x, \blacktriangle, x = a), (x', \blacktriangleright, \perp)$
<i>Output propagation</i>			
(4)	$(x, y, z), (x', \blacktriangle, z')$	$\xrightarrow{\text{true}}$	$(x, y, z'), (x', \blacktriangle, z')$ for $y \in \{\blacktriangleleft, \blacktriangleright\}$
(5)	$(x, y, \top), (x', y', \perp)$	$\xrightarrow{\text{true}}$	$(x, y, \perp), (x', y', \perp)$ for $y, y' \in \{\blacktriangleleft, \blacktriangleright\}$

By fairness, the first rule must be used until one or zero $\blacktriangle$ remains. Moreover, by fairness, the second and third rules will respectively move the $\blacktriangleleft$'s to the left, and the $\blacktriangleright$'s to the right. If some $(x, \blacktriangle, y)$ remains, then, by fairness and the fourth rule, it will propagate its output y , which is $\top$ iff $x = a$, by the choice of initial states and by rule (2–3). Otherwise, if the population is of even length, the fifth rule will be used to propagate $\perp$.

Figure 1 depicts all configurations reachable from the initial configuration $aab \in L$. Any fair run of a population protocol leads to a bottom strongly connected component of such a reachability graph. Thus, in this example, every fair runs leads to $a_{\blacktriangleleft}^\top a_{\blacktriangle}^\top b_{\blacktriangleright}^\top$, which is $\top$ -stable. Note that $a_{\blacktriangleleft}^\perp a_{\blacktriangle}^\perp b_{\blacktriangleright}^\perp$ is a $\perp$ -consensus, but is not $\perp$ -stable.



■ **Figure 1** Configurations reachable from aab , where x_y^z stands for (x, y, z) . Self-loops arising from “no operation” transitions are omitted. The hatched nodes are consensus; the bottom one is stable.

Further observe that the reachability graph of Figure 1 has non-trivial cycles, for instance:

$$a_{\blacktriangleleft}^{\perp} a_{\blacktriangleleft}^{\top} b_{\blacktriangleleft}^{\perp} \rightarrow a_{\blacktriangleleft}^{\perp} a_{\blacktriangleleft}^{\top} b_{\blacktriangleleft}^{\top} \rightarrow a_{\blacktriangleleft}^{\perp} a_{\blacktriangleleft}^{\top} b_{\blacktriangleleft}^{\perp} \rightarrow a_{\blacktriangleleft}^{\perp} a_{\blacktriangleleft}^{\top} b_{\blacktriangleleft}^{\top} \rightarrow \dots$$

Informally, the first and second agents are fighting to convince the third agent. However, by fairness, this is not allowed to happen indefinitely. Eventually, the configuration $a_{\blacktriangleleft}^{\perp} a_{\blacktriangleleft}^{\top} b_{\blacktriangleleft}^{\top}$, at the bottom, is reached.

In the above specific protocol $\mathcal{P}$, for each initial configuration, the reachability graph has a unique trivial bottom strongly connected component, made of one configuration of the form $\blacktriangleleft^n \blacktriangleleft \blacktriangleleft^n$ or $\blacktriangleleft^n \blacktriangleleft^n$. However, in general, it needs not be unique or trivial. A fair run becomes b -stable iff it visits a bottom strongly connected component whose configurations are *all* b -consensus.

In a decider, for a given initial configuration, *all* bottom strongly connected components must consist only of stable configurations, all of the *same* output.

We now provide a generic upper bound on the complexity of languages decided by population protocols; we will exhibit a matching lower bound in Section 6 for $\text{IO-PP}[+1]$. Recall that $\text{NSPACE}(n)$ is the class of languages recognized by linear-bounded nondeterministic Turing machines, i.e., nondeterministic machines that require space $O(n)$ over inputs of size n . Languages of this class are exactly the context-sensitive languages [22].

► **Theorem 4.** *Let $\mathcal{N}$ be a set of numerical predicates, all of which decidable in $\text{NSPACE}(n)$. We have $\text{PP}[\mathcal{N}] \subseteq \text{NSPACE}(n)$.*

Proof. Recall that a configuration is $\top$ -stable if every reachable configuration from it is a $\top$ -consensus (i.e., all agents have opinion $\top$). Consider a protocol in $\text{PP}[\mathcal{N}]$. The set of configurations that are *not* $\top$ -stable is decidable in $\text{NSPACE}(n)$. Indeed, it is sufficient to nondeterministically guess a partial run (i.e., a finite sequence of configurations $w_0 \rightarrow w_1 \rightarrow \dots \rightarrow w_n$) that leads to a configuration that is not a $\top$ -consensus. Each transition can be guessed, its condition checked, and its effect applied in $\text{NSPACE}(n)$.

By the Immerman–Szelepcsényi theorem, $\text{NSPACE}(n)$ is closed under complement, and so the set of $\top$ -stable configurations is in $\text{NSPACE}(n)$. Since the protocol is a decider, to check that a word w is accepted, it is sufficient to nondeterministically guess a partial run from w , and check that it ends in a $\top$ -stable configuration. These are all tasks in $\text{NSPACE}(n)$. ◀

3 Semantic restrictions of population protocols: a toolbox

In this section, we define two restrictions of population protocols that will be used to simplify our constructions.

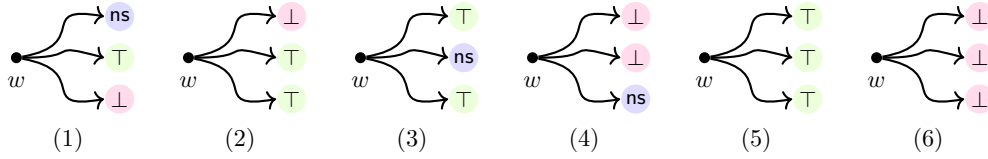
We first define *semi-deciders*, which do not require the full behavior of deciders with respect to stable configurations. Throughout the next sections, we will see that semi-deciders are much easier to define for some languages, and we will rely on the forthcoming Lemma 6 to combine semi-deciders into deciders.

We then define *protocols with stabilizing inputs*, which are protocols where agents can change their mind about their input. Such protocols are harder to design, since they are more robust to change, but we show, in Lemma 11, that they exhibit a strong closure property: they are closed under alphabet rewriting.

3.1 Semi-deciders

► **Definition 5.** We say that a PP *semi-decides* $L \subseteq \Sigma^+$ if for all $u \in \Sigma^+$ and every fair run ρ starting from u , we have $u \in L$ iff ρ contains a $\top$ -stable configuration. We use the terms $\text{PP}[\mathcal{N}]$ -*semi-decidable* and $\text{IO-PP}[\mathcal{N}]$ -*semi-decidable* with the obvious meaning.

Note that a decider is a semi-decider in which we additionally require that $u \notin L$ iff ρ visits a $\perp$ -stable configuration; that is, all fair runs visit a stable configuration. Illustrating the differences, Figure 2 shows the situations that can occur for runs from an input w .



■ **Figure 2** Configurations labeled $\top$, $\perp$ and ns are $\top$ -stable, $\perp$ -stable, and configurations from which no stable configurations are reachable. Arrows depict partial, finite runs. Situations (1-6) can happen in PP. Only situations (4-6) can happen in semi-deciders, while only situations (5-6) can in deciders.

Our naming convention is justified by the following property:



► **Lemma 6.** A language L is $\text{PP}[\mathcal{N}]$ -decidable iff L and its complement are $\text{PP}[\mathcal{N}]$ -semi-decidable. The same holds for $\text{IO-PP}[\mathcal{N}]$.

Proof sketch. We combine the two semi-deciders for L and for $\Sigma^+ \setminus L$ by running them in parallel. Each agent stores a pair of states, one for each semi-decider, together with a *belief* indicating which semi-decider it currently trusts. Transitions either simulate one step of one of the semi-deciders, or *flip* the belief so that beliefs can align; in particular, an agent may flip when it meets an agent with the opposite belief, or when its currently trusted component produces a $\perp$ -witness. For any input u , exactly one of the two simulations eventually provides such a witness (since exactly one of $u \in L$ or $u \notin L$ holds), which forces all agents to converge to the correct belief and yields a stable consensus, hence a decider. ◀

We note these elementary closure properties:



► **Lemma 7.** If $L_1, L_2 \subseteq \Sigma^+$ are $\text{PP}[\mathcal{N}]$ -semi-decidable, then it is also the case for $L_1 \cap L_2$ and $L_1 \cup L_2$. This further holds for $\text{IO-PP}[\mathcal{N}]$ and deciders.

3.2 Protocols with stabilizing inputs

Agents of a protocol are generally not aware that they have reached a stable consensus and hence “terminated.” To carry out a task A and use its output in a subsequent task B, a protocol has to perform *both* tasks concurrently. The protocol for task B thus guesses what the output of task A is going to be, but ought to be able to self-correct if it becomes clear that the guess was wrong. In this subsection, we introduce a formal notion for this “self-correction” which will simplify the design of composable protocols; this is inspired by a recent presentation of [31] for $\text{PP}[\emptyset]$.

Let us consider protocols where each agent keeps a copy of its input. Formally, a protocol $\mathcal{P} = (Q, \Sigma, O, \Delta)$ is said to be *input-saving* if

- $Q = \Sigma \times R$ for some finite set R ;
- Each $\sigma \in \Sigma$ is identified with (σ, r_σ) for some $r_\sigma \in R$; and
- The first component σ of any state $(\sigma, r) \in Q$ is left unchanged by all transitions of Δ .

For all $w \in Q^+$, let $\iota(w) \in \Sigma^+$ be the projection of w onto its first component. Given $u, v \in Q^n$, we write $u \rightsquigarrow v$ if either $u \rightarrow v$, or v equals u except at a single position i where $u[i] = (\sigma, r)$ and $v[i] = (\sigma', r)$. This second type of transitions models a “change of mind” of agent i on its input. We write $\rightsquigarrow^*$ for the reflexive transitive closure of $\rightsquigarrow$. Note what we do not change the definition of run, which still relies on $\rightarrow$ only.

► **Definition 8.** We say that $\mathcal{P}$ semi-decides $L \subseteq \Sigma^+$ *with stabilizing inputs* if (a) $\mathcal{P}$ is input-saving, and (b) for all $u \in \Sigma^+$, all $u \rightsquigarrow^* v$ and every fair run ρ starting from v , it is the case that $\iota(v) \in L$ iff ρ visits a $\top$ -stable configuration. We use the term “*decides*” if (b) is strengthened with the condition that $\iota(v) \notin L$ iff ρ visits a $\perp$ -stable configuration.

Note that this is more robust than simply semi-deciding: if $\mathcal{P}$ semi-decides L with stabilizing inputs, then it semi-decides L . Intuitively, after $u \rightsquigarrow^* v$, the configuration of the population (i.e., the projection of v onto the second component) may be *incompatible* with $\iota(v)$ since agents have possibly changed their mind several times on their input. Computing with stabilizing inputs means that the protocol is able to fix its configuration so that it reflects the expected output on $\iota(v)$.

It is known that any language $L \in \text{PP}[\emptyset]$ can be decided, and hence semi-decided, with stabilizing inputs² [31]. Let us turn to an example, which will be useful later:



► **Proposition 9.** *The language $0^*1^* \dots k^*$ is IO-PP[<]-semi-decidable with stabilizing inputs.*

Proof sketch. Let $\Sigma = \{0, 1, \dots, k\}$ and $L = 0^*1^* \dots k^*$. The protocol $\mathcal{P} = (Q, \Sigma, \Delta, O)$ for L is defined by $Q = \Sigma \times \{\top, \perp\}$, $O((\sigma, o)) = o$, each $\sigma \in \Sigma$ identified with $(\sigma, \top)$, and these rules:

$$\begin{aligned} (x, \top), (y, o) &\xrightarrow{<} (x, \perp), (y, o) && \text{for } x > y, \\ (x, \perp), (y, o) &\xrightarrow{\text{true}} (x, \top), (y, o). \end{aligned}$$

The purpose of the first rule is to detect a misordering. The second rule allows any agent to nondeterministically reset itself. ◀

We now turn to closure properties of protocols with stabilizing inputs. We first cover union and intersection, then move on to (nondeterministic) alphabet rewriting.



► **Lemma 10.** *If $L_1, L_2 \subseteq \Sigma^+$ are $\text{PP}[\mathcal{N}]$ -semi-decidable with stabilizing inputs, then it is also the case for $L_1 \cap L_2$ and $L_1 \cup L_2$. This further holds for $\text{IO-PP}[\mathcal{N}]$ and deciders.*

Given $f: \Sigma \rightarrow 2^\Gamma$ and $w \in \Sigma^n$, let $f(w) = \{w'_1 \cdots w'_n \mid w'_i \in f(w_i) \text{ for each } i \in [1..n]\}$. For example, if $f(0) = \{a, b\}$ and $f(1) = \{b, c\}$, then $f(01) = \{ab, ac, bb, bc\}$. We extend this notion to languages: $f(L) = \bigcup_{w \in L} f(w)$.



► **Lemma 11.** *Let $f: \Sigma \rightarrow 2^\Gamma$. If $L \subseteq \Sigma^+$ is $\text{PP}[\mathcal{N}]$ -semi-decidable with stabilizing inputs, then $f(L)$ is $\text{PP}[\mathcal{N}]$ -semi-decidable with stabilizing inputs. This further holds for $\text{IO-PP}[\mathcal{N}]$.*

Proof sketch. The construction builds a protocol that semi-decides $f(L)$ by simulating, in its second component, the input-saving semi-decider for L on a *guessed* word $v \in \Sigma^+$ compatible with the real input $u \in \Gamma^+$ (i.e., $u \in f(v)$). Agents may revise this guess: whenever a $\perp$ opinion is observed in the simulated component, an agent is allowed to change its guessed letter to any σ consistent with its input letter (preserving $u \in f(v)$ letterwise). If $u \in f(L)$, fairness ensures that the population can eventually rewrite the guessed word into some $v' \in L$, after which the simulation of the semi-decider for L reaches a $\top$ -stable configuration and changes become disabled; if $u \notin f(L)$, reaching a $\top$ -stable configuration would force the simulated input to lie in L , which is not possible. ◀

4 Expressiveness of $\text{IO-PP}[\prec]$

In this section, we provide a precise characterization of the languages decided by $\text{IO-PP}[\prec]$. For any language $L \subseteq \Sigma^*$, let us write $\equiv_L$ for the syntactic congruence of L , i.e., $w \equiv_L w'$ iff $uwv \in L \Leftrightarrow uw'v \in L$ for all $u, v \in \Sigma^*$. A language is in the class **DA** if it is regular, and satisfies, writing $\alpha(w) \subseteq \Sigma$ for the set of letters appearing in a word w :

$$(\forall w \in \Sigma^*) [w \equiv_L w^2 \rightarrow (\forall h \in \alpha(w)^*) [w \equiv_L w \cdot h \cdot w]], \quad (1)$$

It is a fascinating result of Pin and Weil [28] that $\text{DA} = \Delta_2[\prec]$. We will leverage both characterizations to show the theorem below. The left-to-right inclusion will rely on (1), and the converse on $\text{DA} = \Delta_2[\prec]$, semi-deciders, and protocols with stabilizing inputs.

► **Theorem 12.** $\text{IO-PP}[\prec] = \text{DA}$.

4.1 $\text{IO-PP}[\prec] \subseteq \text{DA}$

We first show that the set of stable configurations in a $\text{PP}[\prec]$, and thus in an $\text{IO-PP}[\prec]$, admits a simple description. Given a finite alphabet Σ and two words $u, v \in \Sigma^*$, we say that u is a *subword* of v , written $u \preceq v$, if u can be obtained from v by removing letters. It is well known that $\preceq$ is a *well-quasi-order* over Σ^* [21]. A consequence is that every strictly decreasing sequence of subword-closed sets must be finite.



► **Lemma 13.** *Any protocol in $\text{PP}[\prec]$ is a well-structured transition system w.r.t. $\preceq$, that is, for all $u, u', w \in Q^*$ with $u \preceq w$ and $u \rightarrow u'$, there exists w' such that $u' \preceq w'$ and $w \rightarrow w'$. Further, the set of b -stable configurations is subword-closed and computable.*

As expected from the definition of **DA**, the core of the argument showing the inclusion of $\text{IO-PP}[\prec]$ in **DA** will rely on a pumping argument. As a first observation, we show that, in an $\text{IO-PP}[\prec]$, an infix of the form wzw with $\alpha(z) \subseteq \alpha(w)$ can, in some sense, mimic the behavior of the protocol on the simpler infix w .

² It was claimed earlier by [2], but definitions and proofs were deferred to a full paper that never appeared.



► **Lemma 14.** *In an IO-PP[<], if $uwv \rightarrow^* u'w'v'$ with $|u| = |u'|$ and $|v| = |v'|$, then for all z with $\alpha(z) \subseteq \alpha(w)$, there exists z' such that $uwzvw \rightarrow^* u'w'z'w'v'$ and $\alpha(z') \subseteq \alpha(w')$.*

We use this first result to show a pumping lemma on IO-PP[<]. It results from the fact that the sets of $\top$ -stable and $\perp$ -stable configurations are subword-closed and Lemma 14.



► **Lemma 15.** *Let $L \in \text{IO-PP}[<]$. There exists a computable $m \geq 1$ such that, for all $w_1, \dots, w_m \in \Sigma^+$ and $z \in \Sigma^*$ with $\alpha(w_1) = \dots = \alpha(w_m) \supseteq \alpha(z)$, we have $w_1 \cdots w_m \equiv_L (w_1 \cdots w_m)z(w_1 \cdots w_m)$.*

Proof sketch. Lemma 13 indicates that for both $b \in \{\top, \perp\}$, the set of b -stable configurations is subword-closed. As a consequence, writing B^ε for $B \cup \{\varepsilon\}$, it is a finite union of languages of the form $A_1^* B_1^\varepsilon \cdots A_k^* B_k^\varepsilon$ with A_i, B_i subsets of Q , the set of states of our protocol [20, Sect. 6.1.1]. Let K_b be the maximal factorization size k over all those languages, $K = \max(K_\top, K_\perp)$ and $m = 2K + 1$. Lemma 13 also states that the set of b -stable configurations is computable for both b , in particular we can compute $K_\top, K_\perp$ and m .

Intuitively, if we have a run from $w_1 \cdots w_m$ to a $\top$ -stable configuration $w' = w'_1 \cdots w'_m$, then we can match w' with one of the expressions above, and some w'_i is entirely contained in some A_j^* .

We then use Lemma 14 to expand the run $w \rightarrow w'$ to a run from $w' = w_1 \cdots w_m z w_1 \cdots w_m$ to some b -stable configuration w' , by expanding the w_i section into $w_i \cdots w_m z w_1 \cdots w_i$.

We can thus show that if w is accepted, so is w' . Similarly, we show that if w is not accepted, then it can reach a $\perp$ -stable configuration, and so can w' . We conclude that one is in the language if and only if the other is. To prove the congruence relation we only need to add words u and v around w , which does not significantly alter the proof. ◀

We can now show that IO-PP[<]-decidable languages satisfy the definition of DA. In Lemma 16, we show that Equation 1 is satisfied, and in Lemma 17, that the languages are regular, concluding the proof.

► **Lemma 16.** *Let $L \in \text{IO-PP}[<]$ and let $w \in \Sigma^*$ be such that $w \equiv_L w^2$. It is the case that $w \equiv_L w \cdot h \cdot w$ for all $h \in \alpha(w)^*$.*

Proof. Lemma 15 implies that there is a (computable) number m such that $w^m h w^m \equiv_L w^m$, and since $w \equiv_L w^2$, we obtain $w \equiv_L w \cdot h \cdot w$ as claimed. ◀



► **Lemma 17.** *Every IO-PP[<]-decidable language is regular. Moreover, given an IO-PP[<]-protocol $\mathcal{P}$, we can construct a finite automaton with the same language.*

Proof sketch. We show that beyond some length, every word must contain a pattern of the form $(w_1 \cdots w_m)z(w_1 \cdots w_m)$ with $\alpha(w_1) = \dots = \alpha(w_m) \supseteq \alpha(z)$. By Lemma 15, this means that every sufficiently long word is $\equiv_L$ -equivalent to a shorter one, meaning that $\equiv_L$ has finitely many equivalence classes. Since we can compute m , we can compute this automaton. ◀

4.2 DA $\subseteq$ IO-PP[<]

Recall that $L \in \Pi_2[<]$ iff $\Sigma^* \setminus L \in \Sigma_2[<]$. Since DA is equal to $\Delta_2[<] = \Sigma_2[<] \cap \Pi_2[<]$, it is enough to prove that $\Sigma_2[<]$ languages are IO-PP[<]-semi-decidable, appealing to Lemma 6 to conclude. Note that $\Sigma_2[<]$ is the set of languages expressible as finite unions of languages of the form $L = A_0^* a_1 A_2^* \cdots a_{m-1} A_m^*$ with the A_i 's being subalphabets (see, e.g., [27, Thm. 8.8]).

Since semi-deciders are closed under union by Lemma 7, we need only show that L is $\text{IO-PP}[\leq]$ -semi-decidable to conclude. We start with a technical proposition that extends a result of [31] to immediate-observation protocols, then provide a semi-decider for L in Proposition 19. Since all constructions used here are effective, our proof also implies that given a finite automaton recognizing a DA language, we can effectively construct an $\text{IO-PP}[\leq]$ -protocol recognizing the same language.



► **Proposition 18.** *The language $\{w \in \Sigma^+ \mid |w|_a = 1\}$ is $\text{IO-PP}[\emptyset]$ -semi-decidable with stabilizing inputs.*

Proof sketch. Each agent stores its current input, its last input, and a belief on the output. If the current input of an agent mismatches its former one, then it resets itself to the current one. Each agent eventually stabilizes to a fixed input $(\sigma, \sigma, \cdot)$. The belief component then controls the consensus: an $(a, a, \top)$ agent can turn any $(\sigma, \sigma, \cdot)$ with $\sigma \neq a$ to $\top$, while the presence of any $(\sigma, \sigma, \perp)$ can spread $\perp$ to other agents. Finally, if an a -agent observes another a , it switches to $\perp$; combined with the rule that lets an a -agent reset itself to $\top$, this makes a -agents alternate forever between $\top$ and $\perp$ whenever there are at least two a 's, preventing stabilization in that case. As a result, if the input contains no a then every fair run reaches a $\perp$ -stable consensus; if it contains exactly one a then every fair run reaches a $\top$ -stable consensus; and if it contains at least two a 's then some agent flips its belief forever. ◀

► **Proposition 19.** *The language $L = A_0^* a_1 A_2^* \cdots a_{m-1} A_m^*$ is $\text{IO-PP}[\leq]$ -semi-decidable (with stabilizing inputs).*

Proof. Let $\Gamma = \{0, 1, \dots, m\}$, $K = 0^* 1^* \cdots m^*$ and $K' = \bigcap_{a \in \Gamma, a \text{ odd}} \{w \in \Gamma^+ \mid |w|_a = 1\}$. By Proposition 9, K is $\text{IO-PP}[\leq]$ -semi-decidable with stabilizing inputs. Furthermore, by Proposition 18 and Lemma 10, K' is $\text{IO-PP}[\emptyset]$ -semi-decidable with stabilizing inputs. By Lemma 10, $K \cap K'$ is $\text{IO-PP}[\leq]$ -semi-decidable with stabilizing inputs. Let $f(i) = A_i$ for even i , and $f(i) = \{a_i\}$ otherwise. We are done by Lemma 11 since $L = f(K \cap K')$. ◀

5 Expressiveness of $\text{PP}[\leq]$

The crisp characterization of the previous section ties $\text{IO-PP}[\leq]$ to a wealth of computational models with strikingly different flavors. Chief among them, DA is characterized by a logic, $\Delta_2[\leq]$, and by partially-ordered unambiguous automata. A natural question is thus whether $\text{PP}[\leq]$ also admits such a diverse array of characterizations. We fall short of providing exact characterizations, but offer, in this section, two large classes of languages, one logically-defined and one based on partially-ordered automata, that are $\text{PP}[\leq]$ -decidable.

Our formalisms will involve Presburger arithmetic, the first-order theory of the naturals with order and addition; e.g., $\phi(x) = (\exists y \in \mathbb{N})[y \geq 1 \wedge x = 2y]$ holds iff x is a positive even number. Write $\equiv_c$ for the equivalence of naturals modulo c . It is well known that Presburger arithmetic together with $\equiv_c$ with any $c \geq 2$ admits quantifier elimination. For our purposes, a *Presburger formula* is a (quantifier-free) Boolean combination of predicates of the form $\sum_{i=1}^n a_i x_i < b$ or $\sum_{i=1}^n a_i x_i \equiv_c b$, where $a_i, b \in \mathbb{Z}$, $c \geq 2$ and variables x_i are over $\mathbb{N}$.

5.1 First-order logic over word intervals

► **Definition 20.** For $w \in \Sigma^n$ and $\sigma \in \Sigma$, let $\#_\sigma: \{1, \dots, n\} \times \{1, \dots, n\} \rightarrow \mathbb{N}$ be the function that counts the number of occurrences of σ between two positions of w , i.e., $\#_\sigma(x, y) = |w[x..y]|_\sigma$. We define FO^{int} as first-order logic over word *intervals*, that is, with access to numerical values $\#_a(x, y)$ where x and y are either first-order variables, the first

position (denoted “1”) or the last position (denoted “**max**”). We will study $\text{FO}^{\text{int}}[<, +, \equiv]$, where we allow numerical values to be compared, added, and tested modulo c for any constant c . Note that a variable x can be expressed as $\sum_{\sigma \in \Sigma} \#_{\sigma}(1, x)$, we will thus assume that the atomic formulas only have terms of the form $\#_a(x, y)$, though we write 1 for $\sum_{\sigma \in \Sigma} \#_{\sigma}(1, 1)$, and **max** for $\sum_{\sigma \in \Sigma} \#_{\sigma}(1, \mathbf{max})$. The logics Σ_k^{int} , Π_k^{int} , and Δ_k^{int} are naturally defined.

► **Example 21.** Let us show that the *median language* $\bigcup_{n \in \mathbb{N}} \Sigma^n a \Sigma^n$ belongs to $\Delta_1^{\text{int}}[<, +, \equiv]$. The following predicate asserts that x is the middle position: $\psi(x) = \sum_{\sigma \in \Sigma} \#_{\sigma}(1, x) = \sum_{\sigma \in \Sigma} \#_{\sigma}(x, \mathbf{max})$. The median language can either be expressed by $(\exists x)[\psi(x) \wedge a(x)]$, or by $(\forall x)[(\psi(x) \rightarrow a(x)) \wedge \mathbf{max} \equiv_2 1]$. Note that $a(x)$ is the same as $\#_a(x, x) = 1$.

We now provide a convenient characterization of languages in $\Sigma_1^{\text{int}}[<, +, \equiv]$.



► **Lemma 22.** *Any language from $\Sigma_1^{\text{int}}[<, +, \equiv]$ is a finite union of languages of the form $K = \{a_0 w_1 a_1 \cdots w_m a_m \mid w_i \in \Sigma^*, a_i \in \Sigma, \varphi(x_{i,\sigma} \mapsto |w_i|_{\sigma}, y_{i,\sigma} \mapsto |a_i|_{\sigma})\}$ where $m \geq 0$ and φ is a Presburger formula over variables $\{x_{i,\sigma} \mid i \in [1..m], \sigma \in \Sigma\} \cup \{y_{i,\sigma} \mid i \in [0..m], \sigma \in \Sigma\}$.*

Thanks to the previous lemma, we are able to build a protocol $\text{PP}[<]$ semi-deciding any language in $\Sigma_1^{\text{int}}[<, +, \equiv]$, by building semi-decidars with stabilizing inputs for each K , and then using the closure properties presented in Section 3.2.

► **Proposition 23.** *Any language from $\Sigma_1^{\text{int}}[<, +, \equiv]$ is $\text{PP}[<]$ -semi-decidable.*

Proof. Let $L \in \Sigma_1^{\text{int}}[<, +, \equiv]$. By Lemma 22, L is a finite union of languages of the form $L' = \{a_0 w_1 a_1 \cdots w_m a_m \mid w_i \in \Sigma^*, a_i \in \Sigma, \varphi(x_{i,\sigma} \mapsto |w_i|_{\sigma}, y_{i,\sigma} \mapsto |a_i|_{\sigma})\}$ where $m \geq 0$ and φ is a Presburger formula. It suffices to show that L' is $\text{PP}[\mathcal{N}]$ -semi-decidable with stabilizing inputs. Indeed, by Lemma 10, that class of languages is closed under union.

Let $\underline{\sigma} = \{\sigma \mid \sigma \in \Sigma\}$, $A_i = \underline{\sigma} \times \{i\}$, $W_i = \Sigma \times \{i\}$ and $\Gamma = \bigcup_{i \in [0..m]} A_i \cup \bigcup_{i \in [1..m]} W_i$. We justify that the three following languages K , K' and K'' over alphabet Γ are semi-decided with stabilizing inputs:

- $K = A_0^* W_1^* A_1^* \cdots W_m^* A_m^*$,
- $K' = \{w \in \Gamma^+ \mid \varphi(x_{i,\sigma} \mapsto |w|_{(\sigma,i)}, y_{i,\sigma} \mapsto |w|_{(\underline{\sigma},i)})\}$,
- $K'' = \left\{w \in \Gamma^+ \mid \sum_{\gamma \in A_1} |w|_{\gamma} = 1 \wedge \cdots \wedge \sum_{\gamma \in A_m} |w|_{\gamma} = 1\right\}$ (recall $A_i \cap A_j = \emptyset$ for $i \neq j$).

Let $f(2i) = A_i$ and $f(2i+1) = W_{i+1}$. We have $f(0^* 1^* 2^* \cdots (2m)^*) = K$. By Proposition 9 and Lemma 11, K is $\text{PP}[<]$ -semi-decidable with stabilizing inputs. Since $K', K'' \in \text{PP}[\emptyset]$, these two languages are $\text{PP}[\emptyset]$ -decidable with stabilizing inputs [2, 31].

By Lemma 10, the language $K \cap K' \cap K''$ is $\text{PP}[<]$ -semi-decidable with stabilizing inputs. Let $g((\sigma, i)) = g((\underline{\sigma}, i)) = \{\sigma\}$. We are done by Lemma 11 since $L' = g(K \cap K' \cap K'')$. ◀



► **Corollary 24.** $\Delta_1^{\text{int}}[<, +, \equiv] \subseteq \text{PP}[<]$.

In the forthcoming Conjecture 40, we postulate that these two classes coincide.

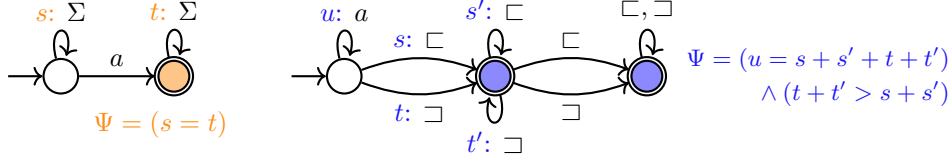
5.2 Partially-ordered Parikh automata

A *partially-ordered nondeterministic Parikh automaton* (poPA) is a tuple $\mathcal{A} = (Q, \Sigma, \delta, q_0, F, \Psi)$ where:

- Q is a finite set of *states* equipped with a partial order $\leq$;
- Σ is a finite *alphabet*;
- $\delta \subseteq Q \times \Sigma \times Q$ is the *transition relation*, that satisfies $p \leq q$ for all $(p, \sigma, q) \in \delta$; if $p = q$, the transition is dubbed a *self-loop*, and otherwise, a *progress transition*;

- $q_0 \in Q$ is the *initial state*, and $F \subseteq Q$ is the set of *final states*;
- Ψ is a Presburger formula over variables δ .

For all $t = (q, \sigma, q') \in \delta$, we write $q \rightarrow^t q'$ and $\pi_\Sigma(t) = \sigma$. We naturally lift these notations to sequences. A word $w \in \Sigma^*$ is *accepted* by $\mathcal{A}$ if there exist $\rho \in \delta^*$ and $q \in F$ such that $q_0 \rightarrow^\rho q$, $\pi_\Sigma(\rho) = w$ and $\Psi(t \mapsto |\rho|_t)$ holds. The *language* of $\mathcal{A}$ is the set $L(\mathcal{A})$ of words it accepts. Figure 3 depicts two examples of poPA.



■ **Figure 3** Example of poPA for the median language $\bigcup_{n \geq 0} \Sigma^n a \Sigma^n$ (left) and the coDyck-witness language $\{a^n v \mid n \geq 1, v \in \{\square, \sqsupset\}^{\geq n}, |v[1..n]|_{\square} > |v[1..n]|_{\sqsupset}\}$ (right).

Lemma 22 allows us to translate a formula from $\Sigma_1^{\text{int}}[<, +, \equiv]$ into a poPA by guessing a factorization $a_0 w_1 a_1 \cdots w_m a_m$ with a progress transition for each a_i , and a self-loop for each w_i ; and then using the Presburger acceptance formula of the automaton to verify the guess. We can also translate a poPA into a formula of $\Sigma_1^{\text{int}}[<, +, \equiv]$ by guessing the position of the progress transitions and verifying the validity of the path. This yields:



► **Theorem 25.** *A language is recognized by some poPA iff it belongs to $\Sigma_1^{\text{int}}[<, +, \equiv]$.*

Recall that $\Sigma_2[<]$ is characterized by partially-ordered automata, while $\Delta_2[<]$ is characterized by *unambiguous* partially-ordered automata. We explore a similar notion for $\Delta_1^{\text{int}}[<, +, \equiv]$ and poPA. A poPA $\mathcal{A}$ is *weakly unambiguous*³ if every $w \in L(\mathcal{A})$ is accepted by at most one path (w.r.t. F and Ψ). Note that the automata of Figure 3 are weakly unambiguous and can be complemented. By Theorem 25, this means that both languages belong to $\Delta_1^{\text{int}}[<, +, \equiv]$. More generally, we conjecture that weakly-unambiguous poPA are closed under complement.



► **Observation 26.** *If weakly-unambiguous poPA are closed under complement, then any language recognized by a weakly-unambiguous poPA belongs to $\Delta_1^{\text{int}}[<, +, \equiv]$.*

In the forthcoming Conjecture 40, we postulate that these two classes coincide. We note, as a sanity check, that since $\Delta_2[<]$ is characterized by unambiguous partially-ordered automata, $\text{IO-PP}[<] = \text{DA} = \Delta_2[<] \subsetneq \Delta_1^{\text{int}}[<, +, \equiv]$. The strictness of the inclusion is in particular witnessed by the *regular* language $\{a^{2^n} \mid n \geq 1\}$ over alphabet $\{a\}$, which trivially belongs to $\Delta_1^{\text{int}}[<, +, \equiv]$ with formula $\mathbf{max} \equiv_2 0$.

► **Remark 27** (Two-way models). Recall that DA is also characterized by two-way *deterministic* partially-ordered automata. We can show that two-way poPA are equivalent to poPA. However, defining two-way deterministic PA as two-way deterministic automata with a Presburger constraint, it is known that the model is as expressive as unambiguous PA [16], which cannot express the languages of Figure 3.

³ The nomenclature, introduced in [7], stems from prior studies [9] which called “unambiguous” the PA with an unambiguous underlying automaton. We note that the examples of Figure 3 can be shown, using the tools of [9], not to be expressible with unambiguous PA.

5.3 The regular languages of $\text{PP}[\prec]$

Lemma 22 gives a clear form for the languages of $\Sigma_1^{\text{int}}[\prec, +, \equiv]$. If the language K therein is regular, this points to the formula φ not arithmetically linking the w_i 's between one another nor imposing nonregular constraints on any single w_i . Hence we have naturally:

► **Conjecture 28.** *Let Com be the set of commutative regular languages. Let $\text{Pol}(\text{Com})$ be the set of languages that are finite unions of languages $L_0 a_1 L_1 \cdots a_n L_n$ with $L_i \in \text{Com}$ and a_i letters. We conjecture that the regular languages of $\Sigma_1^{\text{int}}[\prec, +, \equiv]$ are exactly $\text{Pol}(\text{Com})$.*

The class $\text{Pol}(\text{Com})$ has been studied in previous works: [1] conjectures that they correspond to the regular languages requiring $O(\log n)$ communication and [18] shows that it is included in the largest class of languages, closed under the so-called positive variety operations, that does *not* contain $(ab)^+$. We recall, for contrast, that DA is the largest class of *aperiodic* regular languages, closed under the variety operations, that does not contain $(ab)^+$. To give additional credence to our conjecture, we show:



► **Proposition 29.** *The language $(ab)^+$ does not belong to $\Sigma_1^{\text{int}}[\prec, +, \equiv]$.*

The class $\text{UPol}(\text{Com})$ is defined similarly as $\text{Pol}(\text{Com})$, except that we require that every word w in the marked concatenation has a *unique* decomposition $w = w_0 a_1 w_1 \cdots a_n w_n$ with $w_i \in L_i$. It is known [29] that $\text{UPol}(\text{Com}) = \text{Pol}(\text{Com}) \cap \overline{\text{Pol}(\text{Com})}$. We naturally postulate, in Conjecture 40, that the regular languages of $\text{PP}[\prec]$ are exactly $\text{UPol}(\text{Com})$.

6 Expressiveness of $\text{IO-PP}[\mathcal{N}]$ and $\text{PP}[\mathcal{N}]$ when successor is available

We now consider protocols where transitions may be fired only when two agents are adjacent. Formally, we look at $\text{IO-PP}[\mathcal{N}]$ and $\text{PP}[\mathcal{N}]$ with $+1 \in \mathcal{N}$, where $+1$ (sometimes written **succ** in the literature) is the predicate $\{(x, x+1) \mid x \in \mathbb{N}\}$. This study is reminiscent of the classic study of $\text{FO}^2[\prec, +1]$ [15, 23, 36, 39] that followed that of $\text{FO}^2[\prec]$, the two-variable fragment of $\text{FO}[\prec]$ that is as expressive as $\text{IO-PP}[\prec]$.

► **Example 30.** Recall that the set of all b -stable configurations of a $\text{PP}[\prec]$ is subword-closed, and hence regular. The following $\text{PP}[+1]$ has a non-regular set of $\top$ -stable configurations:

$$\begin{array}{lll} a^{\bar{x}}, b^{\bar{x}} \xrightarrow{\text{true}} a^{\bar{y}}, b^{\bar{y}} & a^{\bar{y}}, a \xrightarrow{+1} \checkmark, a^{\bar{x}} & a^{\bar{y}}, b^{\bar{y}} \xrightarrow{+1} \mathbf{x}, \mathbf{x} \\ & b, b^{\bar{y}} \xrightarrow{+1} b^{\bar{x}}, \checkmark & \end{array}$$

Let us set the opinion of each state to $\top$, except for $\mathbf{x}$. The protocol is constructed so that from a configuration of the form $a^{\bar{x}} a^* b^* b^{\bar{x}}$, one marker moves to the right along the a 's, while the other synchronously moves to the left along the b 's. If they meet at the frontier between a 's and b 's, it means there are as many a 's and b 's, and that state $\mathbf{x}$ appears.

The set of $\top$ -stable configurations cannot be regular since, from $w \in a^{\bar{x}} a^* b^* b^{\bar{x}}$, the state $\mathbf{x}$ will appear iff $|w|_a = |w|_b$, e.g.,

$$a^{\bar{x}} a b b^{\bar{x}} \rightarrow a^{\bar{y}} a b b^{\bar{y}} \rightarrow \checkmark a^{\bar{x}} b b^{\bar{y}} \rightarrow \checkmark a^{\bar{x}} b^{\bar{x}} \checkmark \rightarrow \checkmark a^{\bar{y}} b^{\bar{y}} \checkmark \rightarrow \checkmark \mathbf{x} \checkmark \checkmark.$$

The main result of this section is:

► **Theorem 31.** *Let $\mathcal{N}$ be a set of $\text{NSPACE}(n)$ -decidable numerical predicates that contains $+1$. We have $\text{IO-PP}[\mathcal{N}] = \text{PP}[\mathcal{N}] = \text{NSPACE}(n)$.*

To show this result, it is enough to prove that $\text{NSPACE}(n) \subseteq \text{IO-PP}[+1]$, since $\text{PP}[\mathcal{N}] \subseteq \text{NSPACE}(n)$ has been established beforehand in Theorem 4.

6.1 $\text{NSPACE}(n) \subseteq \text{IO-PP}[+1]$

We start by showing that transitions using the successor predicate can be simulated with immediate-observation transitions.



► **Lemma 32.** *Let L be a $\text{PP}[+1]$ -semi-decidable language. There exists a $\text{PP}[+1]$ -protocol $\mathcal{P}'$ semi-deciding L and such that each $+1$ -transition of $\mathcal{P}'$ is immediate-observation.*

Proof sketch. The construction turns a $\text{PP}[+1]$ -protocol $\mathcal{P}$ into a new $\text{PP}[+1]$ -protocol $\mathcal{P}'$ by replacing each transition $\delta = (a, b) \xrightarrow{+1} (c, d)$ with a short *four-step handshake* along the successor relation, using auxiliary markers a^δ and $b^{\text{ack}, \delta}$. The four rules are:

$$\begin{array}{ll} a, b \xrightarrow{+1} a^\delta, b & a^\delta, b \xrightarrow{+1} a^\delta, b^{\text{ack}, \delta} \\ a^\delta, b^{\text{ack}, \delta} \xrightarrow{+1} c, b^{\text{ack}, \delta} & c, b^{\text{ack}, \delta} \xrightarrow{+1} c, d \end{array}$$

Intuitively, one agent announces that it wants to perform δ , the neighbor acknowledges, and the two agents then update one after the other; additional clean-up rules erase incomplete handshakes so that executions cannot block when several handshakes overlap. ◀



► **Lemma 33.** *The language of a linear-bounded nondeterministic Turing machine is $\text{IO-PP}[+1]$ -semi-decidable.*

Proof sketch. Given a linear-bounded nondeterministic Turing machine M , we construct an $\text{IO-PP}[+1]$ -protocol $\mathcal{P}$ that simulates M under stabilizing inputs. Thanks to Lemma 32, we only need to ensure that **true**-transitions are immediate-observation.

Let Σ' be the tape alphabet of M . The protocol $\mathcal{P}$ works over Γ^* where $\Gamma = (P \cup \{-\}) \times \Sigma' \times \{\text{fst}, -, \text{lst}\}$. For example, $w = (-, a, \text{fst})(p, a, -)(-, b, \text{lst})$ represents the configuration of M in state p , with the head on the second cell, and the tape containing aab .

The protocol $\mathcal{P}$ simulates M using $+1$ -transitions with the agent containing the current state p . If M accepts, then $\mathcal{P}$ spreads $\top$ across the population. There are two challenges:

1. Since we want to semi-decide under *stabilizing inputs*, $\mathcal{P}$ must reset the population and the simulation of M whenever an agent changes its mind on its input. This is implemented as follows. Any agent who has changed its mind can flag the last agent. Using $+1$ -transitions, this can spread from right to left, resetting the population along the way.
2. If the input of $\mathcal{P}$ is an invalid configuration of M , then the population should not reach a stable $\top$ -consensus. For example, if two agents both represent the head of M , then they change their belief to $\perp$ upon meeting. Similarly, we must detect whether “fst” and “lst” occur exactly in the first and last agent.

The resulting protocol $\mathcal{P}$ does not semi-decide the language of M , but rather the set of configurations of M leading to acceptance. So, we need to intersect with the set of initial configurations, and then project onto the second component of Γ . This can be done thanks to Propositions 9 and 18 and Lemmas 10 and 11. ◀

► **Corollary 34.** $\text{NSPACE}(n) \subseteq \text{IO-PP}[+1]$.

Proof. Let $L \in \text{NSPACE}(n)$. As $\text{NSPACE}(n)$ is closed under complement, there exist two linear-bounded nondeterministic Turing machines M and $\bar{M}$ recognizing L and $\Sigma^+ \setminus L$, respectively. By Lemma 33, we can build two $\text{IO-PP}[+1]$ -protocols, say $\mathcal{P}_M$ and $\mathcal{P}_{\bar{M}}$, that semi-decide L and $\Sigma^+ \setminus L$, respectively. Lemma 6 allows to conclude. ◀

7 Decidability of checking whether a population protocol is a decider

A good portion of our results assumes that a given population protocol is a decider or a semi-decider in order to *construct* an object (a Turing machine in Theorem 4, a formula in Theorem 12, etc.). To fully understand how constructive these proofs are, we ought to study whether it is decidable, given a population protocol, to check if it is a decider. In other words, *is the syntax of deciders decidable?* From the perspective of formal verification, this is also a natural problem, known as the well-specification problem. It is decidable for $\text{PP}[\emptyset]$ [13], and undecidable when protocols are extended with an infinite alphabet (where agents carry data from an infinite domain, and transitions can use equality constraints between those data), except for immediate-observation protocols [37].

We show that the problem is undecidable already for $\text{PP}[\leq]$ and $\text{IO-PP}[+1]$, and provide a natural conjecture that would entail that the problem is decidable for $\text{IO-PP}[\leq]$.

7.1 The syntax of $\text{PP}[\leq]$, $\text{IO-PP}[+1]$ and $\text{PP}[+1]$ deciders are undecidable

First, we consider the *emptiness problem*: given a $\text{PP}[\mathcal{N}]$ protocol $\mathcal{P}$, is there an execution from an initial configuration u to a $\top$ -stable one? We use a technique from [37] to show a general reduction to the syntax problem.



► **Lemma 35.** *The emptiness problem for $\text{PP}[\mathcal{N}]$ (resp. $\text{IO-PP}[\mathcal{N}]$) reduces to deciding the complement of the syntax of $\text{PP}[\mathcal{N}]$ (resp. $\text{IO-PP}[\mathcal{N}]$) deciders.*

Proof sketch. From a protocol $\mathcal{P} = (Q, \Sigma, O, \Delta)$, we construct a protocol $\mathcal{P}'$ such that $\mathcal{P}'$ is not a decider iff $\mathcal{P}$ can reach a $\top$ -stable configuration. The construction adds a fresh sink state $q_\perp$ with $O(q_\perp) = \perp$, and for every state q with $O(q) = \perp$ we add a transition allowing an agent in q to switch to $q_\perp$.

As a consequence, from any configuration that contains a $\perp$ -agent, a fair execution can drive the system to the $\perp$ -stable consensus $q_\perp^*$. On the other hand, if a $\top$ -stable configuration is reachable in $\mathcal{P}$, then the same configuration is reachable in $\mathcal{P}'$ and remains $\top$ -stable there: none of the added transitions to $q_\perp$ are enabled from $\top$ -states. Therefore $\mathcal{P}'$ is not a decider.

Conversely, if $\mathcal{P}$ has no reachable $\top$ -stable configuration, then no fair run of $\mathcal{P}'$ stabilizes to $\top$. Every fair run stabilizes to $q_\perp^*$ and $\mathcal{P}'$ decides the empty language. The construction preserves immediate observation, as each new transition updates at most one agent. ◀

This reduction, combined with the translation from linear-bounded Turing machines to $\text{IO-PP}[+1]$ from Section 6, already yields the following result.



► **Corollary 36.** *The syntax of $\text{IO-PP}[+1]$ and $\text{PP}[+1]$ deciders are undecidable.*



► **Theorem 37.** *The emptiness problem is undecidable for $\text{PP}[\leq]$. Hence the syntax of $\text{PP}[\leq]$ deciders is also undecidable.*

Proof sketch. We reduce from the Post correspondence problem. Specifically, we take two homomorphisms $h_1, h_2: B^* \rightarrow A^*$ and construct a $\text{PP}[\leq]$ that can reach a $\top$ -stable configuration if and only if there is a word u such that $h_1(u) = h_2(u)$. We obtain it as a product of three protocols. One makes sure that we can only reach a $\top$ -stable configuration from an initial one of the form $\#_1 v \#_2 u \#_3$ with $v \in B^*$ and $u \in A^*$. The two others check that we can reach a $\top$ -stable configuration only when $h_1(u) = v$ and $h_2(u) = v$, respectively. Each one does so by making agents simulate two reading heads going through u and v in lockstep and verifying that $h_i(u) = v$. Initially all agents have opinion $\perp$, and in order to

change it they must carry the reading head at some point, hence we cannot skip any letter in order to reach a $\top$ -stable configuration.

Hence, a $\top$ -stable configuration is reachable iff there exist u, v with $h_1(u) = v = h_2(u)$. ◀

7.2 The syntax of IO-PP[<] deciders is decidable, conditionally

We present a conjecture on the reachability relation of IO-PP[<], then show that it entails decidability of the syntax of their deciders:

► **Conjecture 38.** *The set of configurations reachable from a DA language in an IO-PP[<] protocol is also a DA language.*



► **Theorem 39.** *If Conjecture 38 holds, then the syntax of IO-PP[<] deciders is decidable.*

Proof sketch. We use two semi-decision procedures. The first one checks, for each length n , if the protocol is a decider on words of length n . If the input protocol is not a decider, we will observe it for some n . The second one looks for invariants witnessing that the protocol is a decider. We look for two disjoint languages $K_\top, K_\perp$ such that every input word is in one of the two, they are closed under transitions of the protocol, and from everywhere in K_b we can reach a b -stable configuration. Assuming the conjecture, we can assume that those invariants are in DA. We show that we can enumerate potential regular invariants and check the requirements, yielding the second semi-decision procedure. ◀

In fact, one could slightly strengthen the statement above: it suffices to know that the set of configurations reachable from a DA language in $\mathcal{P}$ (instead of in every IO-PP[<]-protocol) is also a DA language, to be able to check whether $\mathcal{P}$ is a decider. This means that if we can prove Conjecture 38 over only a subclass of IO-PP[<]-protocols, then we know that the syntax of that subclass is decidable.

8 Open questions

The most tantalizing open question left open by this work is the characterization of PP[<]. We conjecture that the logic and automata models introduced in Section 5 are tight:

► **Conjecture 40.** *PP[<] is the class of languages recognized by $\Delta_1^{\text{int}}[<, +, \equiv]$ and the class of languages recognized by weakly unambiguous poPA. The regular languages of PP[<] are exactly UPol(Com).*

Conjecture 40 would entail in particular that weakly unambiguous poPA are closed under complement. We note that it is not known whether weakly unambiguous PA themselves are closed under complement, but it may be easier to show such closure under the partially ordered assumption.

Conjecture 38 is also very natural and left open.

References

- 1 Anil Ada. On the non-deterministic communication complexity of regular languages. In *Proc. 12th International Conference on Developments in Language Theory (DLT)*, pages 96–107, 2008. doi:10.1007/978-3-540-85780-8_7.
- 2 Dana Angluin, James Aspnes, Melody Chan, Michael J. Fischer, Hong Jiang, and René Peralta. Stably computable properties of network graphs. In *First IEEE International Conference on Distributed Computing in Sensor Systems (DCOSS)*, pages 63–74, 2005. doi:10.1007/11502593_8.

- 3 Dana Angluin, James Aspnes, Zoë Diamadi, Michael J. Fischer, and René Peralta. Computation in networks of passively mobile finite-state sensors. *Distributed Computing*, 18(4):235–253, 2006. doi:10.1007/s00446-005-0138-3.
- 4 Dana Angluin, James Aspnes, David Eisenstat, and Eric Ruppert. The computational power of population protocols. *Distributed Computing*, 20(4):279–304, 2007. doi:10.1007/s00446-007-0040-2.
- 5 James Aspnes and Eric Ruppert. An introduction to population protocols. In *Middleware for Network Eccentric and Mobile Applications*, pages 97–120. Springer, 2009. doi:10.1007/978-3-540-89707-1_5.
- 6 Michael Blondin and François Ladouceur. Population protocols with unordered data. In *Proc. 50th International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 115:1–115:20, 2023. doi:10.4230/LIPICS.ICALP.2023.115.
- 7 Alin Bostan, Arnaud Carayol, Florent Koechlin, and Cyril Nicaud. Weakly-unambiguous Parikh automata and their link to holonomic series. In *Proc. 47th International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 114:1–114:16, 2020. doi:10.4230/LIPICS.ICALP.2020.114.
- 8 Olivier Bournez, Johanne Cohen, and Mikaël Rabie. Homonym population protocols. *Theory of Computing Systems*, 62(5):1318–1346, 2018. doi:10.1007/S00224-017-9833-2.
- 9 Michaël Cadilhac, Alain Finkel, and Pierre McKenzie. Unambiguous constrained automata. *International Journal of Foundations of Computer Science*, 24(7):1099–1116, 2013. doi:10.1142/S0129054113400339.
- 10 Ho-Lin Chen, David Doty, and David Soloveichik. Deterministic function computation with chemical reaction networks. *Natural Computing*, 13(4):517–534, 2014. doi:10.1007/S11047-013-9393-6.
- 11 Zoë Diamadi and Michael J Fischer. A simple game for the study of trust in distributed systems. *Wuhan University Journal of Natural Sciences*, 6(1):72–82, 2001. doi:10.1007/BF03160228.
- 12 Javier Esparza and Michael Blondin. *Automata theory: An algorithmic approach*. The MIT Press, 2023.
- 13 Javier Esparza, Pierre Ganty, Jérôme Leroux, and Rupak Majumdar. Verification of population protocols. In *Proc. 26th International Conference on Concurrency Theory (CONCUR)*, pages 470–482, 2015. doi:10.4230/LIPICS.CONCUR.2015.470.
- 14 Javier Esparza, Pierre Ganty, Rupak Majumdar, and Chana Weil-Kennedy. Verification of immediate observation population protocols. In *Proc. 29th International Conference on Concurrency Theory (CONCUR)*, volume 118, pages 31:1–31:16, 2018. doi:10.4230/LIPICS.CONCUR.2018.31.
- 15 Kousha Etessami, Moshe Y. Vardi, and Thomas Wilke. First-order logic with two variables and unary temporal logic. *Information and Computation*, 179(2):279–295, 2002. doi:10.1006/INCO.2001.2953.
- 16 Emmanuel Filiot, Shibashis Guha, and Nicolas Mazzocchi. Two-way Parikh automata. In *Proc. 39th IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS)*, pages 40:1–40:14, 2019. doi:10.4230/LIPICS.FSTTCS.2019.40.
- 17 Adam Ganczorz, Leszek Gasieniec, Tomasz Jurdzinski, Jakub Kowalski, and Grzegorz Stachowiak. Selective population protocols. In *Proc. 26th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, pages 225–239, 2024. doi:10.1007/978-3-031-74498-3_16.
- 18 Antonio Cano Gómez, Giovanna Guaiana, and Jean-Éric Pin. Regular languages and partial commutations. *Information and Computation*, 230:76–96, 2013. doi:10.1016/J.IC.2013.07.003.
- 19 Rachid Guerraoui and Eric Ruppert. Names trump malice: Tiny mobile agents can tolerate Byzantine failures. In *Proc. 36th International Colloquium Automata, Languages and Programming (ICALP)*, pages 484–495, 2009. doi:10.1007/978-3-642-02930-1_40.

- 20 Simon Halfon. *On Effective Representations of Well Quasi-Orderings. (Représentations Effectives des Beaux Pré-Ordres)*. PhD thesis, Université Paris-Saclay, France, 2018. URL: <https://tel.archives-ouvertes.fr/tel-01945232>.
- 21 Graham Higman. Ordering by divisibility in abstract algebras. *Proceedings of the London Mathematical Society*, s3-2(1):326–336, 1952. doi:10.1112/plms/s3-2.1.326.
- 22 Sige-Yuki Kuroda. Classes of languages and linear-bounded automata. *Information and Control*, 7(2):207–223, 1964. doi:10.1016/S0019-9958(64)90120-2.
- 23 Kamal Lodaya, Paritosh K. Pandya, and Simoni S. Shah. Around dot depth two. In *Proc. 14th International Conference on Developments in Language Theory (DLT)*, pages 303–315, 2010. doi:10.1007/978-3-642-14455-4_28.
- 24 M. Lothaire. *Algebraic Combinatorics on Words*. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 2002.
- 25 Othon Michail, Ioannis Chatzigiannakis, and Paul G. Spirakis. Mediated population protocols. *Theoretical Computer Science*, 412(22):2434–2450, 2011. doi:10.1016/J.TCS.2011.02.003.
- 26 J. Andres Montoya. Asymptotic reasoning with two variables. In *Proc. 31st International Workshop on Logic, Language, Information, and Computation (WoLLIC)*, pages 38–55, 2025. doi:10.1007/978-3-031-99536-1_3.
- 27 Jean-Éric Pin. Syntactic semigroups. In *Handbook of Formal Languages, Volume 1: Word, Language, Grammar*, pages 679–746. Springer, 1997. doi:10.1007/978-3-642-59136-5_10.
- 28 Jean-Éric Pin and Pascal Weil. Polynomial closure and unambiguous product. In *Proc. 22nd International Colloquium on Automata, Languages and Programming (ICALP)*, pages 348–359, 1995. doi:10.1007/3-540-60084-1_87.
- 29 Thomas Place and Marc Zeitoun. All about unambiguous polynomial closure. *TheoretiCS*, 2, 2023. doi:10.46298/THEORETICS.23.11.
- 30 Frank P Ramsey. On a problem of formal logic. In *Classic Papers in Combinatorics*, pages 1–24. Springer, 1987. doi:10.1007/978-0-8176-4842-8_1.
- 31 Michael Raskin. Modular population protocols. In *Proc. 20th International Symposium on Algorithmics of Wireless Networks (ALGOWIN)*, pages 173–187, 2024. doi:10.1007/978-3-031-74580-5_13.
- 32 Marcel-Paul Schützenberger. Sur le produit de concaténation non ambigü. *Semigroup Forum*, 13(1):47–75, 1976. doi:10.1007/bf02194921.
- 33 Thomas Schwentick, Denis Thérien, and Heribert Vollmer. Partially-ordered two-way automata: A new characterization of DA. In *Proc. 5th International Conference on Developments in Language Theory (DLT)*, pages 239–250, 2001. doi:10.1007/3-540-46011-X_20.
- 34 Michael Sipser. *Introduction to the theory of computation*. Thomson Course Technology, second edition, 2006.
- 35 Pascal Tesson and Denis Thérien. Diamonds are forever: The variety DA. In *Semigroups, algorithms, automata and languages*, pages 475–499. World Scientific, 2002. doi:10.1142/5050.
- 36 Denis Thérien and Thomas Wilke. Over words, two variables are as powerful as one quantifier alternation. In *Proc. 30th Annual ACM Symposium on the Theory of Computing (STOC)*, pages 234–240, 1998. doi:10.1145/276698.276749.
- 37 Steffen van Bergerem, Roland Guttenberg, Sandra Kiefer, Corto Mascle, Nicolas Waldburger, and Chana Weil-Kennedy. Verification of population protocols with unordered data. In *Proc. 51st International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 156:1–156:20, 2024. doi:10.4230/LIPICS.ICALP.2024.156.
- 38 Chana Weil-Kennedy. *Observation Petri Nets*. PhD thesis, Technical University of Munich, Germany, 2023.
- 39 Philipp Weis and Neil Immerman. Structure theorem and strict alternation hierarchy for FO² on words. In *Proc. 21st International Workshop on Computer Science Logic (CSL)*, pages 343–357, 2007. doi:10.1007/978-3-540-74915-8_27.

- 40 Hiroto Yasumi, Fukuhito Ooshita, and Michiko Inoue. Population protocols for graph class identification problems. In *Proc. 25th International Conference on Principles of Distributed Systems, (OPODIS)*, pages 13:1–13:19, 2021. doi:10.4230/LIPICS.OPODIS.2021.13.

A

 Appendix

A.1 Missing proofs from Section 3.1



► **Lemma 6.** *A language L is $\text{PP}[\mathcal{N}]$ -decidable iff L and its complement are $\text{PP}[\mathcal{N}]$ -semi-decidable. The same holds for $\text{IO-PP}[\mathcal{N}]$.*

Proof. $\Rightarrow$) This is immediate, since a protocol deciding L is a semi-decider for L , and the complement protocol, obtained by swapping $\top$ and $\perp$ in the opinion function, is a semi-decider for $\Sigma^+ \setminus L$.

$\Leftarrow$) This requires combining the two semi-deciding protocols $\mathcal{P}_+, \mathcal{P}_-$, for L and $\Sigma^+ \setminus L$ respectively, into a protocol $\mathcal{P}$. We let $\mathcal{P}_s = (Q_s, \Sigma, O_s, \Delta_s)$, for $s \in \{+, -\}$, and we define $\mathcal{P} = (Q, \Sigma', O, \Delta)$ as follows.

We let $Q = Q_+ \times Q_- \times \{+, -\}$, with $\Sigma' = \{(a, a, s) \mid a \in \Sigma, s = + \text{ iff } a \in L\}$. We identify $a \in \Sigma$ with the state $(a, a, s) \in Q$ where $s = +$ iff $a \in L$. By this identification, the protocol $\mathcal{P}$ recognizes a language over Σ and is correct on single agent runs by definition (since with a single agent no transition can be taken). The third component is called the *belief* of the agent. The function O maps each (q_+, q_-, s) to $O_+(q_+)$ if $s = +$, and $\neg O_-(q_-)$ otherwise. Finally, the transitions of $\mathcal{P}$ are of two kinds:

- (*Simulation.*) A transition of $\mathcal{P}_+$ or $\mathcal{P}_-$ can be used, on the appropriate component Q_+ or Q_- . The two other components stay the same.
- (*Flip.*) If two agents have different beliefs, then one of them can flip its belief. The **true** numerical predicate is used for these transitions. An agent in state (q_+, q_-, s) can also flip its belief by itself if $O_s(q_s) = \perp$.

Consider $u \in \Sigma^+$ of length at least 2. Assume that $u \in L$, the case where $u \notin L$ being symmetric. Let $v \in \text{Post}^*(u)$. Let v_+ and v_- be the projections of v on Q_+ and Q_- respectively. Observe that for both $s \in \{+, -\}$, v_s is reachable from u in $\mathcal{P}_s$. In consequence, since $u \in L$, there is an execution of $\mathcal{P}_+$ from v_+ to a $\top$ -stable w_+ . There is also an execution of $\mathcal{P}_-$ from v_- to a configuration w_- where at least one agent has opinion $\perp$. By following both those executions one after the other, we can go from v to a configuration w in $\mathcal{P}$, so that the projection of w on Q_+ and Q_- are w_+ and w_- .

- If the agent which has opinion $\perp$ in w_- has opinion $-$, it can switch it to $+$.
- Then, we can flip all the other agents to $+$, by making them observe this agent. The resulting configuration is $\top$ -stable.

As mentioned, the case $u \notin L$ is symmetric, showing that $\mathcal{P}$ is a decider. Furthermore, observe that if both $\mathcal{P}_+$ and $\mathcal{P}_-$ are *immediate observation* protocols, then so is $\mathcal{P}$. ◀



► **Lemma 7.** *If $L_1, L_2 \subseteq \Sigma^+$ are $\text{PP}[\mathcal{N}]$ -semi-decidable, then it is also the case for $L_1 \cap L_2$ and $L_1 \cup L_2$. This further holds for $\text{IO-PP}[\mathcal{N}]$ and deciders.*

Proof. The proof is standard, and we will give, in Lemma 10, a slightly more intricate proof with the same ideas, so we omit it here. ◀

A.2 Missing proofs from Section 3.2



► **Proposition 9.** *The language $0^*1^* \dots k^*$ is $\text{IO-PP}[\prec]$ -semi-decidable with stabilizing inputs.*

Proof of correctness. Note that $\mathcal{P}$ is input-saving and immediate-observation. Let us show that $\mathcal{P}$ semi-decides L with stabilizing inputs. Let $u \in \Sigma^+$ and $u \rightsquigarrow^* v$. Let $v = w_0 \rightarrow w_1 \rightarrow \dots$ be a fair run. For the special case of $|v| = 1$, note that each state starts with output $\top$ and this remains so as no transition is ever enabled. Let us assume that $|v| \geq 2$.

Case $\iota(v) \in L$. The first rule is permanently disabled in w_0 . Hence, by fairness, the second rule must swap the last component of each agent to $\top$. Once this happens, the population has reached a $\top$ -consensus, and the configuration cannot change anymore.

Case $\iota(v) \notin L$. Recall that the first component of each agent is fixed from w_0 onwards. Let $i < j$ be positions such that $\iota(v)[i] > \iota(v)[j]$. Using the second rule, all agents can change their opinion to $\top$, and then, using the first rule, agent i can change its opinion to $\perp$. By fairness, this happens infinitely often, which means that the run does not stabilize. $\blacktriangleleft$



► **Lemma 10.** *If $L_1, L_2 \subseteq \Sigma^+$ are $\text{PP}[\mathcal{N}]$ -semi-decidable with stabilizing inputs, then it is also the case for $L_1 \cap L_2$ and $L_1 \cup L_2$. This further holds for $\text{IO-PP}[\mathcal{N}]$ and deciders.*

Proof. Let $\mathcal{P}_1 = (Q_1, \Sigma, \Delta_1, O_1)$ and $\mathcal{P}_2 = (Q_2, \Sigma, \Delta_2, O_2)$ be the protocols that respectively $\text{PP}[\mathcal{N}]$ -semi-decide L_1 and L_2 with stabilizing inputs.

Intersection. We simply take the product of both protocols; simulate them independently with a common input component; and output the conjunction of their outputs.

Let $Q_1 = \Sigma \times R_1$ and $Q_2 = \Sigma \times R_2$. Formally, we define $\mathcal{P} = (Q, \Sigma, \Delta, O)$ by $R = R_1 \times R_2$, $Q = \Sigma \times R$, $O((\sigma, q_1, q_2)) = O_1((\sigma, q_1)) \wedge O_2((\sigma, q_2))$, and the following rules:

$$\begin{aligned} (\sigma, q_1, r), (\sigma', q_2, s) &\xrightarrow{P} (\sigma, q_3, r), (\sigma', q_4, s) && \text{for } (\sigma, q_1), (\sigma', q_2) \xrightarrow{P} (\sigma, q_3), (\sigma', q_4) \in \Delta_1 \\ &&& \text{and } r, s \in R_2, \\ (\sigma, r, q_1), (\sigma', s, q_2) &\xrightarrow{P} (\sigma, r, q_3), (\sigma', s, q_4) && \text{for } (\sigma, q_1), (\sigma', q_2) \xrightarrow{P} (\sigma, q_3), (\sigma', q_4) \in \Delta_2 \\ &&& \text{and } r, s \in R_1. \end{aligned}$$

We identify each $\sigma \in \Sigma$ with (σ, σ, σ) .

Note that $\mathcal{P}$ is input-saving, and immediate-observation if it is the case of $\mathcal{P}_1$ and $\mathcal{P}_2$. Let us show that $\mathcal{P}$ semi-decides $L = L_1 \cap L_2$ with stabilizing inputs. For every $w \in Q^+$, let $\pi_1(w) \in Q_1^+$ be the projection of w onto its first two components, and let $\pi_2(w) \in Q_2^+$ be the projection of w onto its first and third components. Let $u \in \Sigma^+$ and $u \rightsquigarrow^* v$. Let $v = w_0 \rightarrow w_1 \rightarrow \dots$ be a fair run from v . By definition of $\mathcal{P}$, the sequence $\pi_i(w_0), \pi_i(w_1), \dots$ is a fair run of $\mathcal{P}_i$. Moreover, $\iota(v) = \iota(\pi_1(v)) = \iota(\pi_2(v))$.

Case $\iota(v) \in L$. Since $\iota(v) \in L_1 \cap L_2$, the runs of $\mathcal{P}_1$ and $\mathcal{P}_2$ eventually stabilize to $\top$ -stable configurations. Since O is a conjunction, the same holds for the run of $\mathcal{P}$.

Case $\iota(v) \notin L$. Let $i \in \{1, 2\}$ be such that $\iota(v) \notin L_i$. The run of $\mathcal{P}_i$ visits infinitely many configurations containing a state $q \in Q_i$ with $O_i(q) = \perp$. Since O is a conjunction, the same holds for the run of $\mathcal{P}$. Furthermore, if $\mathcal{P}_i$ is a decider, then the run of $\mathcal{P}_i$ eventually stabilizes to $\perp$ -stable configurations, which implies the same for the run of $\mathcal{P}$.

Union. We cannot simply take the previous construction and redefine the output mapping to $O((\sigma, q_1, q_2)) = O_1((\sigma, q_1)) \vee O_2((\sigma, q_2))$. For example, if $\mathcal{P}_1$ and $\mathcal{P}_2$ have two agents with outputs $\top\perp$ and $\perp\top$ respectively, then their disjunction yields $\top\top$, which is incorrect as neither $\mathcal{P}_1$ nor $\mathcal{P}_2$ is in a $\top$ -consensus.

Instead, we extend Q with an extra component $\{1, 2\}$ that indicates which output should be used, and define $O((\sigma, q_1, q_2, i)) = O_i((\sigma, q_i))$. The transitions from Δ simply ignore this

new component. However, we need to add this (immediate-observation) rule:

$$(\sigma, q_1, q_2, i), (\sigma', q'_1, q'_2, i') \xrightarrow{\text{true}} (\sigma, q_1, q_2, 3-i), (\sigma', q'_1, q'_2, i') \\ \text{for } i \neq i' \vee O_i(q_i) = \perp \vee O_i(q'_i) = \perp.$$

This way, as long as agents have not agreed on a common choice, or as long as their choice is not in a $\top$ -consensus, they may change their mind.

Note that this is correct even for “deciders”: If $\mathcal{P}_1$ and $\mathcal{P}_2$ both reach $\perp$ -stability, then agents will indefinitely change their choice, but nonetheless steadily output $\perp$. $\blacktriangleleft$



► **Lemma 11.** *Let $f: \Sigma \rightarrow 2^\Gamma$. If $L \subseteq \Sigma^+$ is $\text{PP}[\mathcal{N}]$ -semi-decidable with stabilizing inputs, then $f(L)$ is $\text{PP}[\mathcal{N}]$ -semi-decidable with stabilizing inputs. This further holds for $\text{IO-PP}[\mathcal{N}]$.*

Proof. Let $\mathcal{P} = (Q, \Sigma, \Delta, O)$ be the protocol that semi-decides L with stabilizing inputs. Since $\mathcal{P}$ is input-saving, its states are of the form $Q = \Sigma \times R$.

We provide a protocol $\mathcal{P}'$ where, on input $u \in \Gamma^+$, the population internally starts with an arbitrary word v such that $u \in f(v)$ and runs $\mathcal{P}$ on v . With luck, it may be the case that $v \in L$, but this needs not be the case. Therefore, the population may change its choice: If an agent of $\mathcal{P}'$ has input γ and encounters a $\perp$ -state of $\mathcal{P}$, then it can change its internal input of $\mathcal{P}$ to any letter σ such that $\gamma \in f(\sigma)$.

Let us now describe the protocol formally. For every $\gamma \in \Gamma$, let σ_γ be an arbitrary letter⁴ of Σ such that $\gamma \in f(\sigma_\gamma)$. To handle the corner case of populations with a single agent, if there is choice such that $\sigma_\gamma \in L$, then we take one. We define $\mathcal{P}' = (Q', \Gamma, \Delta', O')$ as follows:

- $Q' = \Gamma \times Q$;
- We identify each $\gamma \in \Gamma$ with (γ, σ_γ) ;
- $O'((\gamma, q)) = O(q)$;
- The transitions of Δ' are defined by

$$(\gamma, q_1), (\gamma', q_2) \xrightarrow{P} (\gamma, q_3), (\gamma', q_4) \quad \text{for } \gamma, \gamma' \in \Gamma \text{ and } (q_1, q_2) \xrightarrow{P} (q_3, q_4) \in \Delta, \\ (\gamma, q), (\gamma', q') \xrightarrow{\text{true}} (\gamma, \sigma), (\gamma', q') \quad \text{for } \gamma, \gamma' \in \Gamma, q, q' \in Q \text{ and } \sigma \in \Sigma \text{ such that} \\ \gamma \in f(\sigma), \text{ and } O(q) = \perp \text{ or } O(q') = \perp.$$

The protocol is input-saving. The second rule is immediate-observation. Moreover, if $\mathcal{P}$ is immediate-observation, then it is also the case of the first rule. It remains to show that $\mathcal{P}'$ semi-decides $f(L)$ with stabilizing inputs. For all $w \in (Q')^+$, let $\pi(w)$ be the projection of w onto its second component. Recall that $\iota(w)$ is the projection of w onto its first component. Let $u \in \Gamma^+$ and $u \rightsquigarrow^* v$. Let $v = w_0 \rightarrow w_1 \rightarrow \dots$ be a fair run. The case where $|v| = 1$, and hence $\iota(v) = \gamma \in \Gamma$, is trivially correct by the choice of σ_γ . Thus, suppose that $|v| \geq 2$.

Case $\iota(v) \in f(L)$. By hypothesis, there exists $v' \in L$ such that $\iota(v) \in f(v')$. For the sake of contradiction, suppose that no $\top$ -stable configuration is visited by the fair run. By assumption, there exist indices $i_0 < i_1 < \dots$ such that $O'(w_{i_j}) = \perp$ for all $j \geq 0$. By definition of O' , this means that $O(\pi(w_{i_j})) = \perp$ for all $j \geq 0$. By fairness and the second rule, we can change the second component of the population to v' (and hence the input of $\mathcal{P}$ to v'). Since $\mathcal{P}$ semi-decides L with stabilizing inputs, we can reach a $\top$ -stable configuration of $\mathcal{P}$ in the second component. Consequently, the second rule becomes permanently disabled, which implies that $\mathcal{P}'$ visits a $\top$ -stable configuration, a contradiction.

⁴ If no such σ_γ exists, then we can simply map γ to a dummy state with opinion $\perp$, as no word containing γ belongs to $f(L)$. That being said, we will never invoke the lemma with such an f .

Case $\iota(v) \notin f(L)$. For the sake of contradiction, suppose that there exists $j \geq 0$ such that $O'(w_j) = O'(w_{j+1}) = \dots = \top$. This means that $w_j \rightarrow w_{j+1} \rightarrow \dots$ only uses the first rule. By definition of O' , we have $O(\pi(w_j)) = O(\pi(w_{j+1})) = \dots = \top$. Since $\mathcal{P}$ semi-decides L with stabilizing inputs, this implies that $\iota(\pi(w_j)) \in L$ and hence $\iota(w_j) \in f(L)$, which is a contradiction. $\blacktriangleleft$

A.3 Missing proofs from Section 4.1



► **Lemma 13.** *Any protocol in $\text{PP}[\prec]$ is a well-structured transition system w.r.t. $\preceq$, that is, for all $u, u', w \in Q^*$ with $u \preceq w$ and $u \rightarrow u'$, there exists w' such that $u' \preceq w'$ and $w \rightarrow w'$. Further, the set of b -stable configurations is subword-closed and computable.*

Proof. In a $\text{PP}[\prec]$, we have $u \rightarrow u'$ if and only if there exist $a, b, c, d \in Q$ and $u_1, u_2, u_3 \in Q^*$ such that $u = u_1 a u_2 b u_3$, $u' = u_1 c u_2 d u_3$ and $(a, b) \xrightarrow{\prec} (c, d) \in \Delta$. Let $w \in Q^*$ be such that $u \preceq w$. We can write w as $w = w_1 a w_2 b w_3$ with $u_1 \preceq w_1$, $u_2 \preceq w_2$ and $u_3 \preceq w_3$. We obtain $w \rightarrow w' = w_1 c w_2 d w_3$ and $u' \preceq w'$ as desired.

Let $U_0 = Q^* \setminus O^{-1}(b)^*$, the set of configurations which are *not* b -consensuses. For all i , let $U_{i+1} = U_i \cup \{u \in Q^* \mid (\exists v \in U_i)[u \rightarrow v]\}$, the set of configurations that can reach U_0 in at most $i+1$ steps. We let $D_i = Q^* \setminus U_i$. The set of b -stable configurations is $D = Q^* \setminus \bigcup_{i \in \mathbb{N}} U_i$.

A key observation is that since our only predicate is $<$, all D_i are subword-closed. As a consequence, so is their intersection D . Furthermore, the sequence $(D_i)_{i \in \mathbb{N}}$ is a descending chain of subword-closed sets. Since $\preceq$ is a well quasi-order, every such chain eventually stabilizes, hence there exists $i \in \mathbb{N}$ such that $D_i = D_{i+1}$. We can compute D_i for each index until we find one such that $D_i = D_{i+1}$. Such D_i is then the set of b -stable configurations. $\blacktriangleleft$

We first show that our protocols are robust to the insertion of repeated letters, under specific contexts:



► **Lemma 14.** *In an $\text{IO-PP}[\prec]$, if $uwv \rightarrow^* u'w'v'$ with $|u| = |u'|$ and $|v| = |v'|$, then for all z with $\alpha(z) \subseteq \alpha(w)$, there exists z' such that $uwzvw \rightarrow^* u'w'z'w'v'$ and $\alpha(z') \subseteq \alpha(w')$.*

Proof. We prove this for a single step, the result follows immediately by induction. Suppose there is a step from uwv to $u'w'v'$, let $(a, b) \xrightarrow{\prec} (a', b')$ be the associated transition. Since the protocol is IO, we have $a = a'$ or $b = b'$. We assume $a = a'$, the other case being symmetric.

If the observing agent is in u or v , then $w = w'$ and we can go from $uwzvw$ to $u'w'z'w'v' = u'wzwv'$ with the same transition. If the observing agent is in w , then $u = u'$, $v = v'$ and, from $uwzvw$, we start by applying that transition for each b in z , observing the same agent labeled a as in the step from uwv to $u'w'v'$. We thus turn the factor z into z' where all b are turned into b' . We can then apply that same transition twice to turn each of the two copies of w to w' . Note that $\alpha(z') = (\alpha(z) \cup \{b'\}) \setminus \{b\}$, and $\alpha(w') \supseteq (\alpha(w) \cup \{b'\}) \setminus \{b\} \supseteq \alpha(z')$ as w' is obtained from w by deleting an occurrence of b and adding an occurrence of b' . We obtain $uw'z'w'v'$, which is equal to $u'w'z'w'v'$ (since here $u = u'$ and $v = v'$) with $\alpha(z') \subseteq \alpha(w')$. $\blacktriangleleft$



► **Lemma 15.** *Let $L \in \text{IO-PP}[\prec]$. There exists a computable $m \geq 1$ such that, for all $w_1, \dots, w_m \in \Sigma^+$ and $z \in \Sigma^*$ with $\alpha(w_1) = \dots = \alpha(w_m) \supseteq \alpha(z)$, we have $w_1 \dots w_m \equiv_L (w_1 \dots w_m)z(w_1 \dots w_m)$.*

Proof. Lemma 13 indicates that for both $b \in \{\top, \perp\}$, the set of b -stable configurations is subword-closed. As a consequence, writing B^ε for $B \cup \{\varepsilon\}$, it is a finite union of languages of the form $A_1^* B_1^\varepsilon \dots A_k^* B_k^\varepsilon$ with A_i, B_i subsets of Q , the set of states of our protocol [20, Sect. 6.1.1]. Let K_b be the maximal factorization size k over all those languages, $K = \max(K_\top, K_\perp)$ and

$m = 2K + 1$. Lemma 13 also states that the set of b -stable configurations is computable for both b , in particular we can compute $K_\top, K_\perp$ and m .

Let $w_1, \dots, w_m, z \in \Sigma^*$ be such that $\alpha(w_1) = \dots = \alpha(w_m) \supseteq \alpha(z)$, and let $u, v \in \Sigma^*$. We show that for both $b \in \{\top, \perp\}$, if we can reach a b -stable configuration from $uw_1 \dots w_mv$ then it is also possible from $uw_1 \dots w_mzw_1 \dots w_mv$. This implies that $uw_1 \dots w_mv$ is accepted if and only if $uw_1 \dots w_mzw_1 \dots w_mv$ is, proving the lemma.

Let $b \in \{\top, \perp\}$, suppose we can reach a b -stable configuration w' from $uw_1 \dots w_mv$. Since steps are length-preserving, we can divide w' into $u'w'_1 \dots w'_mv'$ where u', v' and each w'_i have the same length as u, v and each w_i , respectively.

Since w' is b -stable, there exists $k < K_b$ and an expression $A_1^*B_1^\varepsilon \dots A_k^*B_k^\varepsilon$ whose language contains w' , and only contains b -stable words. Since $m > 2K_b \geq 2k$, there exist i, j such that $u'w'_1 \dots w'_{i-1} \in A_1^*B_1^\varepsilon \dots A_j^*$, $w'_{i+1} \dots w'_mv' \in A_j^*B_j^\varepsilon \dots B_k^\varepsilon$, and $w'_i \in A_j^*$.

Since $w = uw_1 \dots w_mv \rightarrow^* u'w'_1 \dots w'_mv' = w'$, by Lemma 14 there is a partial run from

$$\tilde{w} := (uw_1 \dots w_{i-1})w_i(w_{i+1} \dots w_m)z(w_1 \dots w_{i-1})w_i(w_{i+1} \dots w_mv)$$

to

$$\tilde{w}' := (u'w'_1 \dots w'_{i-1})w'_iz'w'_i(w'_{i+1} \dots w'_mv')$$

for some z' with $\alpha(z') \subseteq \alpha(w'_i)$, and $|z'| = |(w_{i+1} \dots w_m)z(w_1 \dots w_{i-1})|$.

Furthermore, since we have $u'w'_1 \dots w'_{i-1} \in A_1^*B_1^* \dots A_j^*$, and $w'_{i+1} \dots w'_mv' \in A_j^*B_j^\varepsilon \dots B_k^\varepsilon$, and $z' \in \alpha(w'_i)^* \subseteq A_j^*$, we infer $\tilde{w}' \in A_1^*B_1^\varepsilon \dots A_k^*B_k^\varepsilon$, hence $\tilde{w}'$ is b -stable.

We have shown that if there is an execution from $uw_1 \dots w_mv$ to a b -stable configuration, then there is also one from $uw_1 \dots w_mzw_1 \dots w_mv$ to a b -stable configuration, for all $u, v \in \Sigma^*$ and both $b \in \{\top, \perp\}$. By definition of the language of $\mathcal{P}$, this means that $w_1 \dots w_m$ and $w_1 \dots w_mzw_1 \dots w_m$ are equivalent with respect to $\equiv_L$. ◀



► **Lemma 17.** *Every IO-PP[<]-decidable language is regular. Moreover, given an IO-PP[<]-protocol $\mathcal{P}$, we can construct a finite automaton with the same language.*

Proof. We start by showing that some patterns, reminiscent of *sesquipowers* [24], are unavoidable in long strings. An m -reducible pattern is a word $w_1 \dots w_mzw_1 \dots w_m$ such that $\alpha(w_1) = \dots = \alpha(w_m) \supseteq \alpha(z)$. We have:

▷ **Claim 41.** For all alphabet Q of size ℓ and all $m \in \mathbb{N}$, there exists $B(\ell, m) \in \mathbb{N}$ such that for all $w \in Q^*$, if $|w| \geq B(\ell, m)$ then w contains an m -reducible pattern.

Proof. We proceed by induction on the size ℓ of the alphabet Q . If Q is a singleton, then the result is clear with $B(1, m) = 2m + 1$.

Otherwise, define $B(\ell, m) = mB(\ell - 1, m)\ell^{mB(\ell-1, m)}$. Let $w \in Q^*$ be such that $|w| \geq B(\ell, m)$, let u be an infix of w of maximal length such that $|\alpha(u)| < \ell$, and let $M = |u|$.

- If $M \geq B(\ell - 1, m)$, then by induction hypothesis u contains an m -reducible pattern, thus so does w .
- Otherwise, we can split w into $r = B(\ell, m)/B(\ell - 1, m) = m\ell^{mB(\ell-1, m)}$ factors of length $B(\ell - 1, m)$, plus a suffix v : $w = u_1 \dots u_rv$. Since $M < B(\ell - 1, m)$, by definition of M , each u_k contains all letters in Q . Since $r \geq m\ell^{mB(\ell-1, m)}$, there must exist indices i, j such that $i + m < j$ and $u_i \dots u_{i+m-1} = u_j \dots u_{j+m-1}$. As all u_k contain the same set of letters, we obtain an m -reducible pattern $(u_i \dots u_{i+m-1})(u_{i+m} \dots u_{j-1})(u_j \dots u_{j+m-1})$. ◀

By Claim 41 applied with m taken from Lemma 15, there is a bound B such that every word of length more than B is equivalent under $\equiv_L$ to a shorter word. As a consequence, every word is equivalent to a word of length at most B .

This means that $\equiv_L$ has finitely many equivalence classes, yielding the result by the Myhill-Nerode theorem. To construct an automaton for this language from an IO-PP[<]-protocol, it suffices to have one state for each word of length at most B (computable by Lemma 15 and Claim 41). For each of those words w and each letter a , there is an a -transition from w to wa if it has length $\leq B$. Otherwise, wa has length $B + 1$ and contains an m -reducible pattern. Let v be the word obtained by reducing it. The a -transition from wa goes to v . The empty word marks the initial state, and final states are those marked by words accepted by the IO-PP[<]-protocol, which can be checked by exploring the finite configuration space reachable from them using the protocol. ◀

A.4 Missing proofs from Section 4.2



► **Proposition 18.** *The language $\{w \in \Sigma^+ \mid |w|_a = 1\}$ is IO-PP[$\emptyset$]-semi-decidable with stabilizing inputs.*

Proof. We define $\mathcal{P} = (Q, \Sigma, \Delta, O)$ by $Q = \Sigma \times \Sigma \times \{\top, \perp\}$, $O((\sigma, \sigma', o)) = (o \wedge \sigma = \sigma') \vee ((\sigma = a \neq \sigma'))$, and the following rules:

$$(\sigma, a, o), q \xrightarrow{\text{true}} (\sigma, \sigma, \perp), q \quad \text{for } \sigma \neq a, \quad (2)$$

$$(a, \sigma, o), q \xrightarrow{\text{true}} (a, a, \top), q, \quad (3)$$

$$(\sigma, \sigma, o), (a, a, \top) \xrightarrow{\text{true}} (\sigma, \sigma, \top), (a, a, \top) \quad \text{for } \sigma \neq a, \quad (4)$$

$$(\sigma, \sigma, o), (\sigma', \sigma', \perp) \xrightarrow{\text{true}} (\sigma, \sigma, \perp), (\sigma', \sigma', \perp), \quad (5)$$

$$(a, a, o), (a, a, o') \xrightarrow{\text{true}} (a, a, \perp), (a, a, o'). \quad (6)$$

We identify a with $(a, a, \top)$, and each $\sigma \in \Sigma \setminus \{a\}$ with $(\sigma, \sigma, \perp)$.

Note that $\mathcal{P}$ is input-saving and immediate-observation. Let us show that $\mathcal{P}$ decides the language of the statement with stabilizing inputs. Let $u \in \Sigma^+$ and $u \rightsquigarrow^* v$. Let $v = w_0 \rightarrow w_1 \rightarrow \dots$ be a fair run.

If $|v| = 1$, then v is of the form $(a, a, \top)$, $(a, \sigma', \perp)$, $(\sigma, a, \top)$, or $(\sigma, \sigma', \perp)$ where $\sigma, \sigma' \neq a$. By definition of O , the first (resp. last) two states have output $\top$ (resp. $\perp$) as desired.

Suppose $|v| \geq 2$. By fairness and rules (2)–(3), we can assume without loss of generality that each agent has its first two components equal and immutable along the fair run.

Case $|\iota(v)|_a = 0$. Since each $\sigma \neq a$ is identified with $(\sigma, \sigma, \perp)$, and since an a can only disappear via rule (2), there must exist a position i and $\sigma \neq a$ such that $v[i] = (\sigma, \sigma, \perp)$. By fairness and rule (5), a $\perp$ -consensus can be reached. From there, all rules are disabled.

Case $|\iota(v)|_a = 1$. The single agent whose first component is a can set its third component to $\top$ with rule (3), and set the third component of other agents to $\top$ with rule (4). From there, all rules are disabled and hence a $\top$ -stable configuration has been reached.

Case $|\iota(v)|_a \geq 2$. By fairness, rules (3) and (6) cause the third component of a -agents to alternate indefinitely between $\top$ and $\perp$. ◀

A.5 Missing proofs from Section 5.1



► **Lemma 22.** *Any language from $\Sigma_1^{\text{int}}[<, +, \equiv]$ is a finite union of languages of the form $K = \{a_0 w_1 a_1 \dots w_m a_m \mid w_i \in \Sigma^*, a_i \in \Sigma, \varphi(x_{i,\sigma} \mapsto |w_i|_\sigma, y_{i,\sigma} \mapsto |a_i|_\sigma)\}$ where $m \geq 0$ and φ is a Presburger formula over variables $\{x_{i,\sigma} \mid i \in [1..m], \sigma \in \Sigma\} \cup \{y_{i,\sigma} \mid i \in [0..m], \sigma \in \Sigma\}$.*

Proof. Let $\varphi = (\exists x_1, \dots, x_m)[\psi] \in \Sigma_1^{\text{int}}[<, +, \equiv]$ where ψ is quantifier-free. Without loss of generality, ψ may assume that $1 = x_0 < x_1 < \dots < x_m < x_{m+1} = \mathbf{max}$. Indeed,

- For the first and last positions, we add the two extra variables with these two constraints, and replace each other occurrence of 1 and **max** by x_0 and x_{m+1} ;
- For the ordering, we can change the formula so that it tests all orderings:

$$\bigvee_{\pi \in S_m} (\exists x_0, x_1, \dots, x_m, x_{m+1})[x_1 \leq \dots \leq x_m \wedge \psi_\pi],$$

where S_m is the set of permutations over $[1..m]$, and ψ_π is ψ with x_i replaced by $x_{\pi(i)}$;

- For distinctness, we can similarly enumerate all possible equivalence classes of “=” and associate a variable to each class, e.g., if we guess that $x_0 < x_1 = x_2 < x_3 = x_4 < x_5$, then we change the subformula to $(\exists y_0, y_1, y_2, y_3)[\psi']$ where ψ' is the formula obtained by renaming variables in ψ as follows: $x_0 \mapsto y_0$, $\{x_1, x_2\} \mapsto y_1$, $\{x_3, x_4\} \mapsto y_2$ and $x_5 \mapsto y_3$.

Once variables are strictly ordered, we can further assume that each expression $\#_a(x_i, x_j)$ satisfies $j \in \{i, i+1\}$. Indeed, if $i < j$, then $\#_\sigma(x_i, x_j)$ can be substituted with

$$\sum_{\ell=i}^{j-1} \#_\sigma(x_\ell, x_{\ell+1}) - \sum_{\ell=i+1}^{j-1} \#_\sigma(x_\ell, x_\ell).$$

Let ψ' denote the formula ψ where each expression $\#_\sigma(x_i, x_{i+1})$ is replaced with $x_{i,\sigma}$, and each $\#_\sigma(x_i, x_i)$ is replaced with $y_{i,\sigma}$. Consider $K = \{a_0 w_1 a_1 \dots w_n a_m \mid w_i \in \Sigma^*, a_i \in \Sigma, \psi'(x_{i,\sigma} \mapsto |w_i|_\sigma, y_{i,\sigma} \mapsto |a_i|_\sigma)\}$. We claim that $v \models (\exists x_0, \dots, x_m)[\psi]$ iff $v \in K$.

$\Rightarrow$) We take $a_i = v[x_i]$ for each $i \in [0..m]$, and $w_i = v[x_{i-1} + 1..x_i - 1]$ for each $i \in [1..m]$. By the assumptions on ψ , it is the case that $\psi'(x_{i,\sigma} \mapsto |w_i|_\sigma, y_{i,\sigma} \mapsto |a_i|_\sigma)$ holds.

$\Leftarrow$) We have $v = a_0 w_1 a_1 w_2 \dots w_m a_m$ where $w_i \in \Sigma^*$, $a_i \in \Sigma$ and $\psi'(x_{i,\sigma} \mapsto |w_i|_\sigma, y_{i,\sigma} \mapsto |a_i|_\sigma)$ holds. By taking $x_i = |a_0 w_1 \dots a_i|$ for $i \in [0..m]$, $\psi(x_0, \dots, x_m)$ holds. $\blacktriangleleft$



► **Corollary 24.** $\Delta_1^{\text{int}}[<, +, \equiv] \subseteq \text{PP}[<]$.

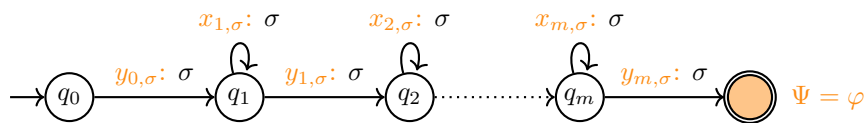
Proof. Let $L \in \Delta_1^{\text{int}}[<, +, \equiv]$. Since $L \in \Sigma_1^{\text{int}}[<, +, \equiv]$, by Proposition 23, L is $\text{PP}[<]$ -semi-decidable. As the complement of L also belongs to $\Sigma_1^{\text{int}}[<, +, \equiv]$, the same applies. Thus, we are done by Lemma 6. $\blacktriangleleft$

A.6 Missing proofs from Section 5.2



► **Theorem 25.** A language is recognized by some poPA iff it belongs to $\Sigma_1^{\text{int}}[<, +, \equiv]$.

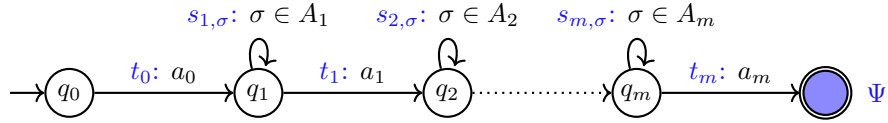
Proof. $\Leftarrow$) Let $L \in \Sigma_1^{\text{int}}[<, +, \equiv]$. By Lemma 22, L can be written as a finite union of languages of the form $K = \{a_0 w_1 a_1 \dots w_m a_m \mid w_i \in \Sigma^*, a_i \in \Sigma, \varphi(x_{i,\sigma} \mapsto |w_i|_\sigma, y_{i,\sigma} \mapsto |a_i|_\sigma)\}$ where $m \geq 0$ and φ is a Presburger formula. Since poPA are nondeterministic, they are trivially closed under union. Thus, it suffices to give a poPA for K , which we do below. Here, each transition labeled with “ σ ” stands for $|\Sigma|$ distinct transitions:



$\Rightarrow$) Let $\mathcal{A}$ be a poPA. Let K be the words of $L(\mathcal{A})$ of length at most one. We can construct a formula for each word of K :

$$\varphi_\varepsilon = \bigwedge_{\sigma \in \Sigma} \#_\sigma(1, \mathbf{max}) = 0 \text{ and } \varphi_a = \#_a(1, \mathbf{max}) = 1 \wedge \bigwedge_{\sigma \neq a} \#_\sigma(1, \mathbf{max}) = 0.$$

Let us now consider $K' = L(\mathcal{A}) \setminus K$. As $\mathcal{A}$ is partially ordered, its language can be described as the finite union of poPA organized as straight lines alternating between self-loops and progress transitions. As we consider K' (and so words of length at least 2), we can explicitly read the first and last letters, which yields automata of the following form, where $a_i \in \Sigma$ and $A_i \subseteq \Sigma$:



We convert such an automaton into this formula:

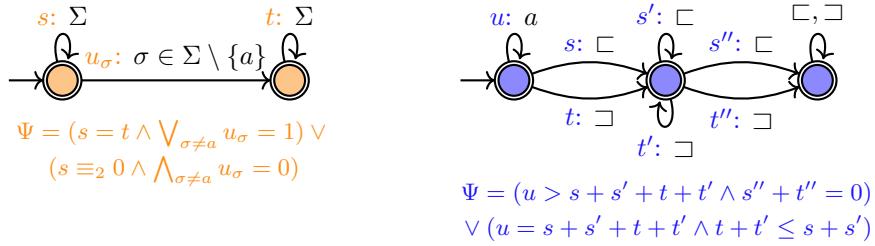
$$(\exists x_0, x_1, \dots, x_m)[1 = x_0 < x_1 < \dots < x_m = \mathbf{max}] \wedge \bigwedge_{i=0}^m \#_{a_i}(x_i, x_i) = 1 \wedge \bigwedge_{i=1}^m \sum_{\sigma \in \Sigma \setminus A_i} \#_\sigma(x_{i-1} + 1, x_i - 1) = 0 \wedge \Psi(t_i \mapsto 1, s_{i,\sigma} \mapsto \#_\sigma(x_{i-1} + 1, x_i - 1)).$$

We are done by taking the disjunction of all formulas. $\blacktriangleleft$



► **Observation 26.** *If weakly-unambiguous poPA are closed under complement, then any language recognized by a weakly-unambiguous poPA belongs to $\Delta_1^{\text{int}}[<, +, \equiv]$.*

Proof. First, observe that the two automata of Figure 3 for the median and coDyck-witness languages can be complemented as follows, while remaining weakly unambiguous:



In general, let $\mathcal{A}$ and $\mathcal{A}'$ be poPA with $L(\mathcal{A}') = \overline{L(\mathcal{A})}$. By Theorem 25, $L(\mathcal{A}), L(\mathcal{A}') \in \Sigma_1[<, +, \equiv]$. Thus, $L(\mathcal{A}) = \overline{L(\mathcal{A}')} \in \Pi_1[<, +, \equiv]$ and so $L(\mathcal{A}) \in \Delta_1[<, +, \equiv]$. $\blacktriangleleft$

A.7 Missing proofs from Section 5.3



► **Proposition 29.** *The language $(ab)^+$ does not belong to $\Sigma_1^{\text{int}}[<, +, \equiv]$.*

Proof. To derive a contradiction, suppose that $(ab)^+$ belongs to $\Sigma_1^{\text{int}}[<, +, \equiv]$. By Lemma 22, $(ab)^+ = K_1 \cup \dots \cup K_\ell$ where K_j is of the form $\{a_{j,0}w_{j,1}a_{j,1} \dots w_{j,m_j}a_{j,m_j} \mid w_{j,i} \in \Sigma^*, a_{j,i} \in \Sigma, \varphi_j(x_{i,\sigma} \mapsto |w_{j,i}|_\sigma, y_{i,\sigma} \mapsto |a_{j,i}|_\sigma)\}$, $m_j \geq 0$ and φ_j is a Presburger formula.

Let $v = (ab)^{n+1}$ where $n = \max(m_1, \dots, m_\ell)$. We have $v \in L$ and so $v \in K_j$ for some $j \in [1..\ell]$. Let $v = a_{j,0}w_{j,1}a_{j,1} \dots w_{j,m_j}a_{j,m_j}$ be decomposed as in K_j . By $|v| = 2n + 2 > 2m_j + 1$

and the pigeonhole principle, there is $i \in [1..m_j]$ with $|w_{j,i}| \geq 2$. Let $w'_{j,i}$ be a word obtained from $w_{j,i}$ by swapping two adjacent letters. Let v' be obtained from v by this change. As the letter counts within $w_{j,i}$ and $w'_{j,i}$ are the same, we have $v' \in K_j$, a contradiction. $\blacktriangleleft$

A.8 Missing proofs from Section 6.1



► **Lemma 32.** *Let L be a PP[+1]-semi-decidable language. There exists a PP[+1]-protocol $\mathcal{P}'$ semi-deciding L and such that each +1-transition of $\mathcal{P}'$ is immediate-observation.*

Proof. Let $\mathcal{P} = (Q, \Sigma, O, \Delta)$ be a PP[+1]-protocol. We construct a new IO-PP[+1]-protocol $\mathcal{P}' = (Q', \Sigma, O', \Delta')$ semi-deciding the same language.

States and outputs. For each transition $\delta = (a, b) \xrightarrow{+1} (c, d)$ in Δ , we introduce auxiliary states a^δ and $b^{\text{ack}, \delta}$. Formally, $Q' = Q \cup \{a^\delta, b^{\text{ack}, \delta} \mid \delta = (a, b) \xrightarrow{+1} (c, d) \in \Delta\}$. The output function ignores the auxiliary markers: $O'(q) = O(q)$, $O'(a^\delta) = O(a)$ and $O'(b^{\text{ack}, \delta}) = O(b)$.

Transitions. The **true**-transitions are left unchanged. Furthermore, for each $\delta = (a, b) \xrightarrow{+1} (c, d) \in \Delta$, the set Δ' contains these four transitions:

$$\begin{aligned} a, b &\xrightarrow{+1} a^\delta, b \\ a^\delta, b &\xrightarrow{+1} a^\delta, b^{\text{ack}, \delta} \\ a^\delta, b^{\text{ack}, \delta} &\xrightarrow{+1} c, b^{\text{ack}, \delta} \\ c, b^{\text{ack}, \delta} &\xrightarrow{+1} c, d. \end{aligned}$$

Each of these steps updates at most one of the two agents at a time, and hence is immediate-observation. To avoid blocking in the presence of overlapping partial simulations, we also include the following “clean-up” rules:

$$\begin{aligned} a^\delta, x &\xrightarrow{+1} a, x && \text{for } x \neq b^{\text{ack}, \delta}, \\ y, b^{\text{ack}, \delta} &\xrightarrow{+1} y, d && \text{for } y \neq a^\delta. \end{aligned}$$

Step simulation.

► **Claim 42.** If $u, v \in Q^*$ and $u \rightarrow v$ in $\mathcal{P}$, then $u \rightarrow^* v$ in $\mathcal{P}'$.

Proof. This is trivial for **true**-transitions. Let $\delta = (a, b) \xrightarrow{+1} (c, d)$ be the transition used in $\mathcal{P}$. Let $u = u_1 a b u_2$ and $v = u_1 c d u_2$. In $\mathcal{P}'$, the four transitions associated with δ yield $u_1 a b u_2 \rightarrow u_1 a^\delta b u_2 \rightarrow u_1 a^\delta b^{\text{ack}, \delta} u_2 \rightarrow u_1 c b^{\text{ack}, \delta} u_2 \rightarrow u_1 c d u_2$. $\blacktriangleleft$

Projection to $\mathcal{P}$. We define a projection $f: Q'^* \rightarrow Q^*$ that interprets a $\mathcal{P}'$ -configuration as a $\mathcal{P}$ -configuration. For $w = w_1 \cdots w_n$, define $f(w) = w'_1 \cdots w'_n$ by:

- if $w_i \in Q$, then $w'_i = w_i$;
- if $w_i = a^\delta$ for some $\delta = (a, b) \xrightarrow{+1} (c, d)$ and $i < n$ with $w_{i+1} = b^{\text{ack}, \delta}$, then $w'_i = c$;
- if $w_i = a^\delta$ but the above condition fails, then $w'_i = a$;
- if $w_i = b^{\text{ack}, \delta}$ for some $\delta = (a, b) \xrightarrow{+1} (c, d)$, then $w'_i = d$.

► **Claim 43.** Let $u \in Q^*$ and $v \in Q'^*$. If $u \rightarrow^* v$ in $\mathcal{P}'$, then $u \rightarrow^* f(v)$ in $\mathcal{P}$.

Proof. We prove the claim by induction on the length of $u \rightarrow^* v$ in $\mathcal{P}'$. The only step that can change the projected configuration is the acknowledgement step $(a^\delta, b) \xrightarrow{+1} (a^\delta, b^{\text{ack}, \delta})$, in which case f precisely applies the corresponding transition $\delta = (a, b) \xrightarrow{+1} (c, d)$ on the projected word. All other rules leave $f(\cdot)$ unchanged. $\blacktriangleleft$

▷ **Claim 44.** For every $v \in Q'^*$ reachable in $\mathcal{P}'$, it is the case that $v \rightarrow^* f(v)$ in $\mathcal{P}'$.

Proof. Starting from v , repeatedly apply the clean-up rules to remove every pending marker a^δ not followed by the matching $b^{\text{ack},\delta}$, and then complete each matching pair $a^\delta b^{\text{ack},\delta}$ by the two remaining simulation steps $(a^\delta, b^{\text{ack},\delta}) \xrightarrow{+1} (c, b^{\text{ack},\delta}) \xrightarrow{+1} (c, d)$. This yields $f(v)$. ◀

We are now ready to prove the lemma. Let us show that $w \in L(\mathcal{P})$ iff $w \in L(\mathcal{P}')$.

⇒) Let $w \rightarrow^* u$ in $\mathcal{P}'$. Thanks to Claim 43, we know that $w \rightarrow^* f(u)$ in $\mathcal{P}$. Since $w \in L(\mathcal{P})$, configuration $f(u)$ can reach a $\top$ -stable configuration v in $\mathcal{P}$. Hence, thanks to Claim 44 and Claim 42, we conclude that $u \rightarrow^* f(u) \rightarrow^* v$ in $\mathcal{P}'$. To derive a contradiction, suppose that v is not $\top$ -stable in $\mathcal{P}'$. Configuration v can reach a non- $\top$ -consensus v' in $\mathcal{P}'$. Consider v' to be such a configuration reachable in a minimal number of steps. The last step must be a transition of the form $(a^\delta, b^{\text{ack},\delta}) \xrightarrow{+1} (c, b^{\text{ack},\delta})$ with $O'(c) = O(c) = \perp$, or $(c, b^{\text{ack},\delta}) \xrightarrow{+1} (c, d)$, with $O'(d) = O(d) = \perp$. Hence, $f(v')$ is not a $\top$ -consensus in $\mathcal{P}$. Therefore, $f(v) = v \rightarrow^* f(v')$ in $\mathcal{P}$ by Claim 43, contradicting the $\top$ -stability of v in $\mathcal{P}$.

⇐) Let $w \rightarrow^* u$ in $\mathcal{P}$. By Claim 42, $w \rightarrow^* u$ in $\mathcal{P}'$. As $w \in L(\mathcal{P}')$, configuration u can reach a $\top$ -stable configuration v in $\mathcal{P}'$. From Claim 44, we have $v \rightarrow^* f(v)$ in $\mathcal{P}'$. Note that $f(v)$ is $\top$ -stable in $\mathcal{P}'$ by construction. Moreover, from Claim 43, we have $u \rightarrow^* f(v)$ in $\mathcal{P}$. To derive contradiction, suppose that $f(v)$ is not $\top$ -stable in $\mathcal{P}$. We have $f(v) \rightarrow^* v'$ in $\mathcal{P}$ for some non- $\top$ -consensus v' . Thanks to Claim 42, we obtain $f(v) \rightarrow^* v'$ in $\mathcal{P}'$, contradicting the $\top$ -stability of $f(v)$ in $\mathcal{P}'$. Hence, $f(v)$ is $\top$ -stable in $\mathcal{P}$, and so $w \in L(\mathcal{P})$. ◀

The rest of the subsection is dedicated to proving the following lemma.



► **Lemma 33.** *The language of a linear-bounded nondeterministic Turing machine is IO-PP[+1]-semi-decidable.*

Let us fix a linear-bounded nondeterministic Turing machine $M = (P, \Sigma, \Sigma', \delta, p_0, p_{\text{acc}}, p_{\text{rej}})$ where P is the set of states; Σ is the input alphabet; $\Sigma' \supseteq \Sigma$ is the tape alphabet; $\delta \subseteq (P \times \Sigma') \times (P \times \Sigma' \times \{\triangleleft, \triangleright\})$ is the transition relation; and $p_0, p_{\text{acc}}, p_{\text{rej}}$ are respectively the initial, accepting and rejecting states.

Traditionally, on input $w \in \Sigma$, a linear-bounded automaton has access to $\vdash w \dashv$; has no left-move on $\vdash$; has no right-move on $\dashv$; and cannot overwrite the endmarkers. For convenience, we consider instead that there are no endmarkers, but that the head stays put when moving left (resp. right) from the first (resp. last) cell. It is simple to construct such a machine with the same language. We further assume that M may only enter p_{acc} with its head on the rightmost cell, and that there is no transition from p_{acc} .

Let $\Gamma = (P \cup \{-\}) \times \Sigma' \times \{\text{fst}, -, \text{lst}\}$. For every $s = (p, a, x) \in \Gamma$, let $\text{sta}(s) = p$ and $\text{pos}(s) = x$. We say that a word $w \in \Gamma^+$, with $|w| \geq 2$, is a *valid configuration* of M if

- $\text{pos}(w[i]) = \text{fst}$ iff $i = 1$,
- $\text{pos}(w[i]) = \text{lst}$ iff $i = |w|$,
- there is a unique position i with $\text{sta}(w[i]) \in P$.

For example, $w = (-, a, \text{fst})(p, a, -)(-, b, \text{lst})$ represents the configuration where M is in state p , with its head on the second cell, and its tape containing aab .

► **Proposition 45.** *Let*

$$L = \{w \in \Gamma^{\geq 2} \mid w \text{ is a valid configuration of } M \text{ and it leads to acceptance}\}.$$

*One can construct PP[+1]-protocol that semi-decides L with stabilizing inputs. Furthermore, its **true**-transitions are immediate-observations.*

Proof. Let us construct a protocol $\mathcal{P} = (Q, \Gamma, O, \Delta)$. We define $Q = \Gamma \times \Gamma \times \Gamma \times \{\perp, \top, \bullet\}$ and $O((x, y, z, o)) = (x = y \wedge o = \top)$. We identify each input $\gamma \in \Gamma$ with $(\gamma, \gamma, \gamma, \perp)$.

Intuitively, the state (x, y, z, o) indicates that the agent has input x ; has been acting as if its input was y ; currently holds z ; and has output belief o . Whenever $x \neq y$, the agent has changed its mind on its input, and must consequently reset the whole population.

The transitions of Δ are defined by these rules:

<i>Simulation of M</i>		
(1)	$(*, *, (p, a, *), *), (*, *, (-, *, *), *) \xrightarrow{+1} (*, *, (-, b, *), *), (*, *, (q, *, *), *)$	for $((p, a), (q, b, \triangleright)) \in \delta$
(2)	$(*, *, (-, *, *), *), (*, *, (p, a, *), *) \xrightarrow{+1} (*, *, (q, *, *), *), (*, *, (-, b, *), *)$	for $((p, a), (q, b, \triangleleft)) \in \delta$
(3)	$(x, *, (p, a, *), *) \xrightarrow{\text{true}} (x, *, (q, b, *), *)$	for $((p, a), (q, b, \triangleright)) \in \delta$ and $\text{pos}(x) = \text{lst}$
(4)	$(x, *, (p, a, *), *) \xrightarrow{\text{true}} (x, *, (q, b, *), *)$	for $((p, a), (q, b, \triangleleft)) \in \delta$ and $\text{pos}(x) = \text{fst}$
<i>Belief propagation</i>		
(5)	$(x, *, *, *), (x', *, z', *) \xrightarrow{\text{true}} (x, *, *, \top), (x', *, z', *)$	if $\text{pos}(x) = \text{fst} \wedge \text{pos}(x') = \text{lst} \wedge \text{sta}(z') = p_{\text{acc}}$
(6)	$(*, *, *, \top), (*, *, *, *) \xrightarrow{+1} (*, *, *, \top), (*, *, *, \top)$	
<i>Reset on input changes</i>		
(7)	$(x, y, *, *), (x', y', *, *) \xrightarrow{\text{true}} (x, y, *, *), (x', y', *, \bullet)$	if $\text{pos}(x') = \text{lst} \wedge (x \neq y \vee x' \neq y')$
(8)	$(*, *, *, *), (x', *, *, \bullet) \xrightarrow{+1} (*, *, *, \bullet), (x', x', x', \perp)$	
(9)	$(x, *, *, \bullet) \xrightarrow{\text{true}} (x, x, x, \perp)$	if $\text{pos}(x) = \text{fst}$
<i>Erroneous configuration detection</i>		
(10)	$(x, *, *, *), (x', *, *, *) \xrightarrow{\text{true}} (x, *, *, *), (x', *, *, \perp)$	if $\text{sta}(x), \text{sta}(x') \in P$
(11)	$(x, *, *, *), (x', *, *, *) \xrightarrow{\text{true}} (x, *, *, *), (x', *, *, \perp)$	if $\text{pos}(x) = \text{pos}(x') \in \{\text{fst}, \text{lst}\}$
(12)	$(x, *, *, *), (x', *, *, *) \xrightarrow{+1} (x, *, *, \perp), (x', *, *, \perp)$	if $\text{pos}(x) = \text{lst} \vee \text{pos}(x') = \text{fst}$

Note that all **true**-transitions are immediate-observation. Let us show that $\mathcal{P}$ semi-decides L with stabilizing inputs. Let $u \in \Gamma^n$. For the special case of $n = 1$, note that each state starts with output $\perp$ and this remains so as no transition is ever enabled. Thus, let us assume that $n \geq 2$.

Let $u = u_0 \rightsquigarrow u_1 \rightsquigarrow \dots \rightsquigarrow u_\ell = v$, and let $v = w_0 \rightarrow w_1 \rightarrow \dots$ be a fair run. Without loss of generality, we may assume that $\ell = 0$ or $\iota(u_\ell) \neq \iota(u_{\ell-1})$. Indeed, it suffices to take ℓ as the last moment where the input changes.

We make a case distinction, where each case assumes that the previous ones do not hold.

Case 1: $\iota(v)$ is a valid configuration. By validity, rules (10–12) are permanently disabled from v onwards.

If $\ell > 0$, then there exists a position i such that $v[i] = (x, y, z, o)$ with $x \neq y$. By validity of $\iota(v)$, we have $\text{pos}(\iota(v)[1]) = \text{fst}$ and $\text{pos}(\iota(v)[n]) = \text{lst}$. Thus, by fairness, the population must eventually use rule (7) to assign $\bullet$ to agent n ; use rule (8) to reset the population from right to left; and use rule (9) to complete this reset with agent 1. From there, rules (7–9) are permanently disabled. Thus, by fairness, M is simulated faithfully with rules (1–4), and $\top$ eventually spreads with rules (5–6).

The case of $\ell = 0$ is the same except for the fact that rules (7–9) are disabled from the very beginning, where the simulation of M already begins.

Case 2: $\iota(v)$ has several “fst”, “lst” or states from P ; or a wrongly placed “fst” or “lst”. Since the first component of each agent stops changing from v onwards, fairness and rules (10–12) guarantee that at least one $\perp$ -belief must occur infinitely often.

Case 3: $\ell = 0$ and $\iota(v)$ is invalid. Rule (5) is never enabled and hence no $\top$ ever appears.

Case 4: $\ell > 0$ and $\iota(v)$ is invalid. By assumption, there is a position i such that $v[i] = (x, y, z, o)$ with $x \neq y$. By the latter, we have $O(v[i]) = \perp$. To obtain a $\top$ -consensus, agent i

must be reset. This requires having at least one $\mathbf{f}\mathbf{s}\mathbf{t}$ in the population. To get rid of all $\mathbf{f}\mathbf{s}\mathbf{t}$, rule (9) must eventually be used. If this happens, then a “fst”-agent now has belief $\perp$. By assumption, this agent must be the first one. The only way to set its belief to $\top$ is by using rule (5), which is impossible by assumption on $\iota(v)$. $\blacktriangleleft$

► **Proposition 46.** *The language $L(M)$ is IO-PP[+1]-semi-decidable.*

Proof. Let K denote the set of valid initial configurations of M . Formally, let $K = \Gamma_0 \Gamma_1^* \Gamma_2$ where $\Gamma_0 = \{p_0\} \times \Sigma \times \{\text{fst}\}$, $\Gamma_1 = \{-\} \times \Sigma \times \{-\}$ and $\Gamma_2 = \{-\} \times \Sigma \times \{\text{lst}\}$.

Let $f(i) = \Gamma_i$ and $U_i = \{w \in \{0, 1, 2\}^+ : |w|_i = 1\}$. We have

$$K = f(0^*1^*2^* \cap U_0 \cap U_2).$$

By Proposition 9, the language $0^*1^*2^*$ is IO-PP[<]-semi-decidable with stabilizing inputs. Replacing “<” by “+1” in its protocol directly yields IO-PP[+1]-semi-decidability with stabilizing inputs. Thus, by Proposition 18 and by Lemmas 10 and 11, the language K is IO-PP[+1]-semi-decidable with stabilizing inputs.

Let $K' = \{w \in \Sigma^{\geq 2} \mid w \text{ is a valid configuration of } M \text{ and it leads to acceptance}\}$. By Proposition 45, there is a PP[+1]-protocol that semi-decides K' with stabilizing inputs and whose **true**-transitions are immediate-observations. Let $g((x, y, z)) = y$. We have

$$L(M) \cap \Sigma^{\geq 2} = g(K \cap K').$$

Thus, by Lemmas 10 and 11, the language $L(M) \cap \Sigma^{\geq 2}$ is PP[+1]-semi-decidable. Moreover, these two lemmas preserve immediate-observation **true**-transitions. Hence, by Lemma 32, the language $L(M) \cap \Sigma^{\geq 2}$ is IO-PP[+1]-semi-decidable.

We can trivially show that $L(M) \cap \Sigma$ is IO-PP[$\emptyset$]-decidable. Thus, we are done by taking the union, with Lemma 7. $\blacktriangleleft$

A.9 Missing proofs from Section 7.1



► **Lemma 35.** *The emptiness problem for PP[$\mathcal{N}$] (resp. IO-PP[$\mathcal{N}$]) reduces to deciding the complement of the syntax of PP[$\mathcal{N}$] (resp. IO-PP[$\mathcal{N}$]) deciders.*

Proof. Take a protocol $\mathcal{P} = (Q, \Sigma, O, \Delta)$. We build a protocol $\mathcal{P}'$ such that $\mathcal{P}'$ is not a decider if and only if a $\top$ -stable configuration is reachable in $\mathcal{P}$.

If Σ contains a letter a with opinion $\top$, then the configuration a is $\top$ -stable and reachable. In that case, we set $\mathcal{P}'$ as an arbitrary protocol which is not a decider.

Otherwise, we build $\mathcal{P}' = (Q', \bar{\Sigma}, O', \Delta')$ from $\mathcal{P}$ as follows: For each initial state $a \in \Sigma$ in $\mathcal{P}$, we add another state $\bar{a}$ with opinion $\perp$. We denote $\bar{\Sigma}$ the set of those additional states. We also add another state $q_\perp$ with opinion $\perp$. The set of initial states of $\mathcal{P}'$ is $\bar{\Sigma}$. For each $\bar{a} \in \bar{\Sigma}$ and $q \in Q'$ we add a transition $(\bar{a}, q) \xrightarrow{\text{true}} (a, q)$. As a consequence, from every initial configuration $\bar{a}_1 \cdots \bar{a}_k$ with $k \geq 2$, we can reach its counterpart in $a_1 \cdots a_k$ in Σ^* .

Further, for every pair of states q_1, q_2 such that at least one of the two has opinion $\perp$, we add a transition $(q_1, q_2) \xrightarrow{\text{true}} (q_1, q_\perp)$. Note that by this last transition, from every configuration of length ≥ 2 which is not a $\top$ -consensus, we can reach a configuration of $q_\perp^*$.

Suppose that there is a run in $\mathcal{P}$ from an initial configuration $u = a_1 \cdots a_k \in \Sigma^+$ to a $\top$ -stable configuration v in $\mathcal{P}$. We must have $k \geq 2$ since we assumed that all $a \in \Sigma$ have opinion $\perp$. In $\mathcal{P}'$, the configuration $\bar{u} = \bar{a}_1 \cdots \bar{a}_k$ is initial, and it can reach both $q_\perp^k$ and v , respectively a $\perp$ and a $\top$ -stable configuration. As a result, the protocol is not a decider.

Now suppose that a $\top$ -stable configuration is unreachable in $\mathcal{P}$. Let u be an initial configuration and let $u \rightarrow^* v$. Since v is not $\top$ -stable, we have $v \rightarrow^* w$ for some configuration

w containing a state with opinion $\perp$. Thus $w \rightarrow^* q_\perp^*$ and the latter is a $\perp$ -consensus. As a result, the protocol is a decider (and recognizes the empty language).

Finally, observe that if $\mathcal{P}$ is immediate-observation then so is $\mathcal{P}'$. $\blacktriangleleft$

↑

► **Corollary 36.** *The syntax of IO-PP[+1] and PP[+1] deciders are undecidable.*

Proof. In Section 6, we presented an effective construction translating a linear-bounded Turing machine into an IO-PP[+1] protocol with the same language. Since emptiness is undecidable for linear-bounded Turing machines [34, Thm. 5.10], it must also be undecidable for IO-PP[+1] protocols. By Lemma 35, so is the syntax of IO-PP[+1] deciders. $\blacktriangleleft$

↑

► **Theorem 37.** *The emptiness problem is undecidable for PP[<]. Hence the syntax of PP[<] deciders is also undecidable.*

Proof. We reduce from the Post correspondence problem (PCP), which is undecidable. We are given two finite alphabets A, B , and two homomorphisms $h_1, h_2: B^* \rightarrow A^*$, and must determine whether there exists $w \in B^*$ such that $h_1(w) = h_2(w)$.

We construct three protocols, each in charge of verifying a property of the initial word:

- $\mathcal{P}_0$ checks that the input word is in $\#_1 A^* \#_2 B^* \#_3$;
- $\mathcal{P}_1$ checks that $h_1(u) = v$ for a word $\#_1 v \#_2 u \#_3$;
- $\mathcal{P}_2$ checks that $h_2(u) = v$ for a word $\#_1 v \#_2 u \#_3$.

For the first one, we simply use the fact that $\#_1 A^* \#_2 B^* \#_3$ is in DA, and hence recognized by an (immediate-observation) protocol $\mathcal{P}_0$. In particular an input word can reach a $\top$ -stable configuration if and only if it is in that language.

We now construct $\mathcal{P}_i$ for $i \in \{1, 2\}$. The set of states of both protocol is

$$\{a, a^{head}, a^{next} \mid a \in A\} \cup B \cup \{(b, \pi) \mid b \in B, \pi \text{ prefix of } h(b)\} \cup \{\#_1, \#_2, \overline{\#_2}, \#_3, q_\top\}.$$

The opinion of $q_\top$ is $\top$, all other states have opinion $\perp$.

Protocol rules are as follows, for all $a, c \in A$, $b, d \in B$ and prefix π of $h_i(b)$:

<i>Start</i>		<i>Head</i>	
$\#_1, a \xrightarrow{\quad} q_\top, a^{head}$		$a^{head}, (b, \pi) \xrightarrow{\quad} a^{next}, (b, \pi a)$	
$\#_2, b \xrightarrow{\quad} \overline{\#_2}, (b, \varepsilon)$		if πa is a prefix of $h_i(b)$	
<i>Next (A)</i>		<i>Next (B)</i>	
$a^{next}, c \xrightarrow{\quad} q_\top, c^{head}$		$(b, h_i(b)), d \xrightarrow{\quad} q_\top, (d, \varepsilon)$	
$a^{next}, \overline{\#_2} \xrightarrow{\quad} q_\top, q_\top$		$(b, h_i(b)), \#_3 \xrightarrow{\quad} q_\top, q_\top$	

Our final protocol $\mathcal{P}$ is simply the product of those three protocols, the opinion of a triple of states being the conjunction of their opinions. This way, we can reach a $\top$ -stable configuration from an initial configuration if and only if we can reach a $\top$ -stable configuration from it in all three protocols.

We now show that this is the case if and only if that initial configuration is of the form $\#_1 u \#_2 v \#_3$ with $u = h_1(v) = h_2(v)$.

If the initial configuration $\#_1 h(a_1) \cdots h(a_k) \#_2 a_1 \cdots a_k \#_3$ satisfies those conditions, then it is straightforward to build an execution from it to $q_\top$ in both $\mathcal{P}_1$ and $\mathcal{P}_2$. Furthermore, it can reach a $\top$ -stable configuration in $\mathcal{P}_0$ since it is in $\#_1 A^* \#_2 B^* \#_3$.

Now consider an initial configuration from which we can reach a $\top$ -consensus in all three protocols. By definition of $\mathcal{P}_0$, it must be of the form $\#_1 a_1 \cdots a_k \#_2 b_1 \cdots b_\ell \#_3$ with $a_1, \dots, a_k \in A$ and $b_1, \dots, b_\ell \in B$.

Now consider protocol $\mathcal{P}_i$, with $i \in \{1, 2\}$. We call the states $\{a^{head}, a^{next} \mid a \in A\} \cup \{\#_1\}$ *special states*. It is easy to see that while we do not use the last **Next** (**A**) rule, there is exactly one special state in the configuration, and there is no special state afterwards. Also observe that only letters to the right of the special state can be modified. Since all agents must reach $q_\top$, this imposes that all agents that have a state in A initially must hold the special state, one by one from left to right.

A symmetric argument shows that all agents with a state in B initially must transform into $q_\top$ using **Next** (**B**) rules, one by one, from left to right. Now let us have a look at the **Head** rule: $a^{head}, (b, \pi) \xrightarrow{} a^{next}, (b, \pi a)$. Let w be the word obtained by taking the letter a associated with each of those transitions, in the order in which they appear in the run.

By the arguments above, we must have $w = a_1 \cdots a_k$, and $w = h_i(b_1) \cdots h_i(b_\ell)$. As a result, the input word is indeed of the form $\#_1 h_i(b_1) \cdots h_i(b_\ell) \#_2 b_1 \cdots b_\ell \#_3$.

As we have shown this for both $i \in \{1, 2\}$, we obtain that the initial configuration must be of the form $\#_1 u \#_2 v \#_3$ with $u = h_1(v) = h_2(v)$.

In conclusion, there is a reachable $\top$ -stable configuration if and only if the PCP instance has a solution. $\blacktriangleleft$

A.10 Missing proofs from Section 7.2



► **Theorem 39.** *If Conjecture 38 holds, then the syntax of IO-PP[<] deciders is decidable.*

Proof. Let $\text{Post}(u) = \{v \mid u \rightarrow v\}$ and $\text{Post}^*(u) = \{v \mid u \rightarrow^* v\}$. Similarly, let $\text{Pre}(u) = \{v \mid v \rightarrow u\}$ and $\text{Pre}^*(u) = \{v \mid v \rightarrow^* u\}$. We extend these to sets, e.g., $\text{Post}^*(L) = \bigcup_{w \in L} \text{Post}^*(w)$.

We reuse the machinery of Section 4.1, specifically Lemma 13, Lemma 15, and the combinatorial Claim 41.

If a IO-PP[<] protocol $\mathcal{P} = (Q, \Sigma, O, \Delta)$ is not a decider, then it is witnessed by the configuration space of words of length n for some $n \geq 2$. This can be checked for each n incrementally. We only have to provide a semi-decision procedure which returns yes if and only if the protocol is a decider.

Observe that assuming Conjecture 38 holds, a protocol $\mathcal{P}$ is a decider if and only if there exist languages $K_\top, K_\perp$ in DA over Q^* such that:

- $K_\top$ and $K_\perp$ are disjoint;
- $\Sigma^* \subseteq K_\top \cup K_\perp$;
- For both b , for all $u \in K_b$ and $v \in \text{Post}(u)$, it is the case that $v \in K_b$;
- For both b , $K_b \subseteq \text{Pre}^*(C_b)$ with C_b the set of b -stable configurations.

The right-to-left direction is clear: if such invariants exist, then the protocol recognizes $K_\top \cap \Sigma^*$. For the other direction, by Theorem 12, if $\mathcal{P}$ is a decider then its language L is in DA, hence so is $\Sigma^* \setminus L$. Take $K_\top = \text{Post}^*(L)$ and $K_\perp = \text{Post}^*(\Sigma^* \setminus L)$. Further, assuming Conjecture 38, $K_\top$ and $K_\perp$ are also in DA. They clearly satisfy all the conditions.

Given $\mathcal{P}$, we enumerate automata $\mathcal{A}, \mathcal{B}_\top, \mathcal{B}_\perp$ recognizing languages in DA.

We check whether:

- $L(\mathcal{A}) \subseteq L(\mathcal{B}_\top)$, $\Sigma^* \setminus L(\mathcal{A}) \subseteq L(\mathcal{B}_\perp)$;
- For both b , for all $u \in L(\mathcal{B}_b)$ and $v \in \text{Post}(u)$, it is the case that $v \in L(\mathcal{B}_b)$;
- For both b , $L(\mathcal{B}_b) \subseteq \text{Pre}^*(C_b)$.

The first condition is easy to check. For the second one, we can build a non-deterministic automaton recognizing $\bigcup_{u \in L(\mathcal{B}_b)} \text{Post}(u)$, by making it guess two positions at which it applies a transition of the protocol, while checking that the resulting word is in $L(\mathcal{B}_b)$. The last condition is more subtle. To check it, we argue that if there is a word in $L(\mathcal{B}_b) \setminus \text{Pre}^*(C_b)$ then there is one of bounded length. It then suffices to check inclusion over words of length below this bound.

First we prove a property of language of DA by a classical use of Ramsey's theorem.

▷ **Claim 47.** For every language $L \subseteq Q^*$ in DA, there exists $m \in \mathbb{N}$, computable from an automaton recognizing L , such that for all $n \geq m$ and $u_1, \dots, u_n, z$ with $\emptyset \neq \alpha(u_1) = \dots = \alpha(u_n) \supseteq \alpha(z)$, we have $u_1 \dots u_n \equiv_L u_1 \dots u_n z u_1 \dots u_n$.

Proof. Let $(M, \cdot, 1_M)$ be the syntactic monoid of L , $\varphi: Q^* \rightarrow M$ a morphism and $F \subseteq M$ such that $L = \varphi^{-1}(F)$.

Given a sequence of words $u_1, \dots, u_n$, consider the complete undirected graph (without loops) over $\{0, \dots, n\}$, where for all $i < j$ the edge $\{i, j\}$ is colored $\varphi(u_{i+1} \dots u_j)$. By Ramsey's theorem [30], there exists a uniform bound $m \in \mathbb{N}$ such that whenever $n \geq m$ this graph contains a monochromatic 3-clique, i.e., there are $i < j < k$ such that $\varphi(u_{i+1} \dots u_j) = \varphi(u_{j+1} \dots u_k) = \varphi(u_{i+1} \dots u_k)$.

Let $u_1, \dots, u_n, z$ with $n \geq m$ and $\emptyset \neq \alpha(u_1) = \dots = \alpha(u_n) \supseteq \alpha(z)$, and let $i < j < k$ be as described above. Since φ is a morphism, $\varphi(u_{i+1} \dots u_k) = \varphi(u_{i+1} \dots u_j) \varphi(u_{j+1} \dots u_k) = \varphi(u_{i+1} \dots u_k)^2$. In other words, $u_{i+1} \dots u_k \equiv_L (u_{i+1} \dots u_k)^2$. By Equation (1), we have $u_{i+1} \dots u_k \equiv_L u_{i+1} \dots u_k (u_{k+1} \dots u_n z u_1 \dots u_i) u_{i+1} \dots u_k$.

By appending a prefix $u_1 \dots u_i$ and a suffix $u_{k+1} \dots u_n$ to both sides of the equivalence, we obtain $u_1 \dots u_n \equiv_L (u_1 \dots u_n) z (u_1 \dots u_n)$. ◀

We are now ready to show the bound on witnesses for (the negation of) the third condition.

▷ **Claim 48.** We can compute a bound k such that if $L(\mathcal{B}_b) \setminus \text{Pre}^*(C_b)$ is non-empty then it contains a word of length at most k .

Proof. Since $L(\mathcal{B}_b)$ belongs to DA, we can compute m from $\mathcal{B}_b$ as given by Claim 47.

By Lemma 13, the set of b -stable configurations is subword-closed and computable. There is therefore a computable k such that it is made of a finite union of languages described by expressions of the form $A_1^* B_1^\epsilon \dots A_\ell^* B_\ell^\epsilon$ with $\ell \leq k$. Let $n = \max(m, k)$.

Let w be a word in $L(\mathcal{B}_b) \setminus \text{Pre}^*(C_b)$ of length $> B(|\Sigma|, n)$ as given by Claim 41. The word w is of the form $x u_1 \dots u_n z u_1 \dots u_n y$ with $\emptyset \neq \alpha(u_1) = \dots = \alpha(u_n) \supseteq \alpha(z)$.

The word $w' = x u_1 \dots u_n y$ belongs to $L(\mathcal{B}_b)$ since $n \geq m$. It is not in $\text{Pre}^*(C_b)$ since otherwise w would be in $\text{Pre}^*(C_b)$, analogously to the proof of Lemma 15. ◀

In conclusion, to check the third condition we can compute the bound k given by the claim above, and check if there is a word of length at most k in $L(\mathcal{B}_b) \setminus \text{Pre}^*(C_b)$. ◀